

C++从入门到精通

刚好遇见你

- **目标读者：**已掌握 C 基础、希望系统学习现代 C++ 的学习者
 - **核心价值：**在 C 基础上掌握 OOP、STL、RAII 与常用现代特性
 - **建议先修：**[C 语言模块](#)
- 这份学习计划按天组织：每天包含**知识点讲解**、**知识点实战例题（含解题思路）**、**作业练习**（含挑战题）；每天文末提供**参考答案**，建议先独立完成再对照。
- 学习建议：**先看当天知识点 → 跟做「知识点实战例题」→ 独立完成作业与挑战 → 对照本节「参考答案」→ 需要更多题再做 [题库](#)。

整体学习计划

第一周：从 C 到 C++ 核心语法

天数	主题	主要内容	参考教程
Day 1	入门与环境	C++ 定位、Hello World、工具链	01 、 02
Day 2	程序结构	头文件、namespace、编译链接	03
Day 3	类型与引用	引用、auto、iostream	04
Day 4	运算符	C++ 扩展、new/delete 初识	05
Day 5	流程控制	if/switch 与 C++ 差异	06
Day 6	循环	传统循环 + 范围 for	07
Day 7	数组/string/vector	string、vector 初探	08

第二周：函数、OOP、STL 与实战

天数	主题	主要内容	参考教程
Day 8	函数	重载、默认参数、inline	09
Day 9	类与对象	class、构造析构、封装	10
Day 10	继承与多态	virtual、override	11
Day 11	模板与 STL	模板、vector/map/algorithm	12
Day 12	智能指针	unique_ptr/shared_ptr、RAII	13
Day 13	综合实战	学生管理系统	14
Day 14	路线回顾	进阶方向与资源	15

附录：专栏全文

Guide 1

C++入门：从C到现代C++

在C语言基础上再进一步

如果你已经学习了 [C语言入门：为什么它是编程的基石](#)，那么你已经掌握了变量、函数、指针这些编程世界的「砖块」。C++ 并不是一门全新的语言——它诞生于 C 语言之上，保留了 C 的高效与灵活，又在此基础上加入了面向对象、泛型编程等强大能力。

今天，让我们一起了解 C++ 的前世今生，以及为什么它在 C 之后值得继续学习。

一、C++ 的前世今生

1.1 诞生背景：C 语言的进化

C++ 由 Bjarne Stroustrup 于 1979 年在贝尔实验室开始设计，最初称为「C with Classes」。1983 年正式命名为 C++（「++」表示在 C 基础上递增、增强）。

历史节点：

- 1979 年：Bjarne Stroustrup 开始设计 C with Classes
- 1983 年：正式命名为 C++
- 1985 年：《The C++ Programming Language》出版
- 1998 年：第一个 ISO 标准 C++98 发布
- 2011 年：C++11 重大更新（智能指针、auto、范围 for 等）
- 2014/2017/2020：C++14、C++17、C++20 持续演进

1.2 设计哲学：C 的效率 + 更高的抽象

C 语言的设计哲学是「相信程序员」，给予极大自由。C++ 继承了这一传统，同时提供了更高层次的抽象工具：

```
#include <iostream>

int main() {
    std::cout << "Hello, C++!" << std::endl;
    return 0;
}
```

与 C 的 `printf` 相比，`std::cout` 是类型安全的流式输出——编译器会在编译期检查类型是否匹配。

1.3 影响力：无处不在的 C++

领域	应用举例
游戏引擎	Unreal Engine、Unity 核心模块
浏览器	Chrome、Firefox 渲染引擎
操作系统	Windows 内核、macOS 部分组件
数据库	MySQL、MongoDB 核心
嵌入式	高性能实时系统
金融	低延迟交易系统

二、C 与 C++ 的关系

2.1 兼容性：C++ 是 C 的超集（几乎）

绝大多数合法的 C 代码可以直接用 C++ 编译器编译（需注意少数差异，如 `void*` 隐式转换）。这意味着你在 C 语言中学到的知识，在 C++ 中依然有效。

特性	C	C++
基本语法	支持	完全兼容
指针与内存	手动管理	支持 + 智能指针（RAII）
组织方式	函数 + 结构体	函数 + 类 + 命名空间
标准库	较小（stdio.h 等）	丰富（STL：vector、map 等）
函数	无重载	支持重载
泛型	无	模板

2.2 C++ 的核心增量特性

- 1. **面向对象**：类、继承、多态，用对象组织复杂逻辑
- 2. **RAII**：资源获取即初始化，用析构函数自动释放资源
- 3. **STL**：标准模板库，提供 vector、string、algorithm 等开箱即用
- 4. **模板与泛型**：编写与类型无关的可复用代码
- 5. **异常处理**：结构化的错误处理机制

运算符、流程控制、循环等基础语法与 C 相同，详见 [C 语言核心语法](#)。本专栏后续章节将聚焦 C++ 独有的扩展。

2.3 为什么学完 C 再学 C++

理由	说明
理解底层	C 让你理解内存与指针；C++ 在此之上构建抽象
避免困惑	先掌握 C 基础，再学 C++ 特性时不会迷失
职业需求	系统编程、游戏开发、高性能计算普遍要求 C/C++
标准演进	现代 C++（C++11 起）大幅简化内存管理与容器使用

三、C++ 的应用场景

3.1 适合用 C++ 的场景

- **性能敏感**：游戏、实时系统、高频交易
- **系统级编程**：操作系统组件、驱动、编译器
- **大型工程**：需要 OOP 与模块化的大型项目
- **跨平台库**：Qt、Boost 等基础设施

3.2 何时继续用 C

- 嵌入式资源极度受限（仅 C 编译器）
- 维护遗留 C 代码库
- 学习计算机底层原理（C 更「纯粹」）

四、第一个 C++ 程序

如果你已完成 C 语言环境配置，通常已安装 g++。下面是一个最小 C++ 程序：

```
#include <iostream>

int main() {
    std::cout << "Hello, C++!" << std::endl;
    std::cout << "从 C 到 C++，继续编程之旅！" << std::endl;
    return 0;
}
```

编译运行：

```
g++ -std=c++17 hello.cpp -o hello
./hello           # Linux/macOS/WSL
hello.exe         # Windows
```

与 C 版本的对比：

```
/* C 版本 */
#include <stdio.h>
int main() {
    printf("Hello, C!\n");
    return 0;
}
```

对比项	C	C++
头文件	stdio.h	iostream
输出	printf	std::cout
命名空间	无	std
源文件扩展名	.c	.cpp

五、知识点实战例题

例题1：最小 Hello

题目：编写程序，输出两行文字：Hello, C++! 与 从 C 到 C++，继续编程之旅！。

解题思路：

- 1. 包含 <iostream>
- 2. 在 main 中用 std::cout 两次输出，注意换行
- 3. 用 g++ -std=c++17 编译运行

完整代码：

```
#include <iostream>

int main() {
    std::cout << "Hello, C++!" << std::endl;
    std::cout << "从 C 到 C++，继续编程之旅！" << std::endl;
    return 0;
}
```

要点/易错：

- 源文件扩展名用 .cpp，编译器用 g++ 而非 gcc

- 忘记 `return 0;` 在 C++ 中对 `main` 通常仍可过编译，但建议写上

例题2：编译命令对比

题目：分别写出编译 `hello.c` 与 `hello.cpp` 的命令，并说明为何扩展名与编译器要匹配。

解题思路：

1. C 源用 `gcc`，C++ 源用 `g++`
2. C++ 建议显式 `-std=c++17`
3. 用错编译器可能导致链接或语言特性错误

完整代码（仅示意两个最小源文件）：

```
/* hello.c */
#include <stdio.h>
int main(void) {
    printf("Hello, C!\n");
    return 0;
}
```

```
/* hello.cpp */
#include <iostream>
int main() {
    std::cout << "Hello, C++!" << std::endl;
    return 0;
}
```

要点/易错：

- `gcc hello.cpp` 有时也能编过，但未按 C++ 正确链接标准库时易踩坑；入门请固定 `g++`
- Windows 下可执行文件多为 `hello.exe`

六、作业练习

先独立完成，做完再到本节文末「参考答案」对照。

练习1：编译运行 Hello C++

1. 创建 `hello.cpp`，写入上文或例题1代码
2. 使用 `g++ -std=c++17 -Wall hello.cpp -o hello` 编译
3. 运行并确认输出正确

练习2：对比 C 与 C++ 输出

分别编写 `hello.c` 和 `hello.cpp`，用 `gcc` 和 `g++` 编译，观察两者输出是否一致。

练习3：简答

- C++ 源文件为什么通常使用 `.cpp` 扩展名？
- `std::cout` 中的 `<<` 是什么运算符？

挑战题：用 `std::cout` 打印一张至少 5 行的「名片」边框（姓名、专业等），边框用 `=` 或 `-`。

七、参考答案

练习1 参考答案

思路/解析：最小可运行程序；`-Wall` 打开常见警告，便于尽早发现笔误。

```
#include <iostream>

int main() {
    std::cout << "Hello, C++!" << std::endl;
    std::cout << "从 C 到 C++, 继续编程之旅!" << std::endl;
    return 0;
}
```

练习2 参考答案

思路/解析：两边都输出问候语即可；关注工具链差异，而不是输出是否「完全一样」。

```
/* hello.c */
#include <stdio.h>
int main(void) {
    printf("Hello\n");
    return 0;
}
```

```
/* hello.cpp */
#include <iostream>
int main() {
    std::cout << "Hello" << std::endl;
    return 0;
}
```

编译：`gcc hello.c -o hello_c`；`g++ -std=c++17 hello.cpp -o hello_cpp`。

练习3 参考答案

思路/解析：

- `.cpp`（以及 `.cc` / `.cxx`）约定告诉工具链接 C++ 编译；扩展名本身不改变语言，但是生态约定。
- `<<` 在这里是**流插入运算符**，把右侧操作数插入到左侧输出流。

挑战题参考答案

思路/解析：先打上下边框，中间逐行输出字段，尽量对齐。

```
#include <iostream>

int main() {
    std::cout << "======" << std::endl;
    std::cout << "| Name:  Xiaoming  |" << std::endl;
    std::cout << "| Major: CS      |" << std::endl;
    std::cout << "| Lang:  C++      |" << std::endl;
    std::cout << "======" << std::endl;
    return 0;
}
```

八、总结与延伸

本节重点回顾

1. C++ 诞生于 C 语言之上，1983 年正式命名，持续演进至 C++20/23
2. C++ 兼容绝大部分 C 语法，并增加 OOP、模板、STL、RAII 等特性
3. 建议先掌握 C 语言基础（至少 01~09 篇），再进入 C++ 专栏
4. C++ 适用于游戏、系统编程、高性能计算等场景

下节预告

下一篇我们将学习《C++ 开发环境搭建：从安装到 Hello World》，配置 g++、VS Code C++ 插件与 CMake 基础。

延伸阅读

- [C 语言入门：为什么它是编程的基石](#)
- [C++ 教程首页](#)
- cppreference.com — C++ 权威参考

📌 知识点卡片

- C++ 由 Bjarne Stroustrup 设计，是 C 语言的扩展与增强
- 核心增量：类、模板、STL、RAII、异常、命名空间
- 源文件扩展名：`.cpp`；编译器：`g++` / `clang++`
- 建议先修：C 语言 01~09 篇

💡 小技巧

- 编译时加上 `-std=c++17 -Wall` 启用现代标准与常见警告
- 初学阶段优先使用 `std::cout` 而非 C 风格 `printf`，体会类型安全

⚠️ 注意事项

- 不要用 C 的思维写 C++（例如过度使用裸指针而忽略 RAII）
- C++ 标准库头文件通常不带 `.h` 后缀（如 `#include <iostream>`）

如果你有任何问题或想法，欢迎在评论区留言讨论！下一篇见！

Guide 2

C++开发环境搭建：从安装到Hello World

在 C 环境之上配置 C++

如果你已按 [C 语言开发环境搭建](#) 配置过 MinGW-w64 或 GCC，那么 **g++ 通常已经可用**——C 与 C++ 共用同一套工具链。本篇重点补充 C++ 特有的配置：g++ 编译选项、VS Code C++ 任务、以及 CMake 简介。

一、C++ 开发环境核心组件

1.1 与 C 环境的对比

组件	C 语言	C++ 额外/差异
编译器	gcc	g++ (或 <code>gcc -x c++</code>)
标准库	C 标准库	C++ 标准库 (iostream、vector 等)
构建工具	Make	CMake (跨平台，推荐)
编辑器	VS Code + C/C++ 插件	相同，tasks.json 改用 g++
调试器	GDB	相同

1.2 验证 g++ 是否已安装

```
g++ --version
```

若已安装 MinGW-w64 或 Linux GCC，应看到类似输出：

```
g++ (MinGW.org GCC-8.2.0-5) 8.2.0
```

若未安装，请参考 [C 语言开发环境搭建](#) 安装 MinGW-w64 (Windows) 或 `sudo apt install g++` (Linux) 。

二、Windows 环境 (C++ 专项)

2.1 VS Code tasks.json (g++ 编译 C++)

在 C 语言配置基础上，将 `gcc` 改为 `g++`，并添加 C++ 标准：

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "g++ build",
      "type": "shell",
      "command": "g++",
      "args": [
        "-std=c++17",
        "-Wall",
        "-g",
        "${file}",
        "-o",

```

```

        "${fileDirname}\\${fileBasenameNoExtension}.exe"
    ],
    "group": {
        "kind": "build",
        "isDefault": true
    },
    "problemMatcher": ["$gcc"]
}
]
}

```

2.2 launch.json (调试 C++)

```

{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "g++ - 调试活动文件",
            "type": "cppvsdbg",
            "request": "launch",
            "program": "${fileDirname}\\${fileBasenameNoExtension}.exe",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${fileDirname}",
            "environment": [],
            "externalConsole": true,
            "preLaunchTask": "g++ build"
        }
    ]
}

```

2.3 安装 VS Code C++ 插件

若尚未安装，搜索并安装 **Extension Pack for C++**（Microsoft 出品），包含 IntelliSense、调试、CMake 支持。

三、macOS / Linux 环境

3.1 安装 g++

macOS:

```

xcode-select --install
clang++ --version # macOS 默认使用 clang++, 兼容 C++

```

Linux (Debian/Ubuntu) :

```

sudo apt update
sudo apt install g++ gdb make cmake

```

3.2 VS Code tasks.json (Unix 路径)

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "g++ build",
      "type": "shell",
      "command": "g++",
      "args": [
        "-std=c++17",
        "-Wall",
        "-g",
        "${file}",
        "-o",
        "${fileDirname}/${fileBasenameNoExtension}"
      ],
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "problemMatcher": ["$gcc"]
    }
  ]
}
```

四、CMake 简介

当项目包含多个 `.cpp` 文件时，手写编译命令会变得繁琐。CMake 是跨平台的构建系统生成器，广泛用于 C++ 项目。

4.1 最小 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.14)
project>HelloCpp)

set(CMAKE_CXX_STANDARD 17)

add_executable(hello hello.cpp)
```

4.2 编译步骤

```
mkdir build && cd build
cmake ..
cmake --build .
./hello          # Linux/macOS
hello.exe        # Windows
```

入门阶段单文件程序用 `g++` 直接编译即可；多文件项目或后续实战章节会用到 CMake。

五、编写第一个 C++ 程序

5.1 创建 hello.cpp

```
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
    std::cout << "你好, C++! " << std::endl;
    return 0;
}
```

5.2 编译运行

终端：

```
g++ -std=c++17 -Wall hello.cpp -o hello
./hello
```

VS Code：按 `Ctrl+Shift+B` 编译，`F5` 调试运行。

5.3 常用 g++ 参数

参数	作用
<code>-std=c++17</code>	指定 C++ 标准（推荐 C++17）
<code>-Wall</code>	开启常见警告
<code>-g</code>	生成调试信息
<code>-O2</code>	优化（发布时使用）
<code>-o name</code>	指定输出文件名

六、常见问题

6.1 找不到 iostream

问题：`fatal error: iostream: No such file or directory`

解决：确认使用 `g++` 而非 `gcc` 编译 `.cpp` 文件。

6.2 中文乱码

解决：源文件保存为 UTF-8；Windows 终端执行 `chcp 65001`。

6.3 C 与 C++ 混用

- `.c` 文件用 `gcc` 编译

- `.cpp` 文件用 `g++` 编译
- 链接时用 `g++` 可自动链接 C++ 标准库

七、知识点实战例题

例题1：命令行编译参数

题目：解释并实际执行：`g++ -std=c++17 -Wall info.cpp -o info`。

解题思路：

1. `-std=c++17` 选择语言标准
2. `-Wall` 打开常见警告
3. `-o info` 指定输出可执行文件名

完整代码（`info.cpp`）：

```
#include <iostream>

int main() {
    std::cout << "C++ toolchain OK" << std::endl;
    return 0;
}
```

要点/易错：

- 漏写 `-o` 时，Linux/macOS 默认生成 `a.out`，Windows MinGW 常生成 `a.exe`
- 源文件名与输出名不要搞反

例题2：两数之和（`iostream`）

题目：从键盘读入两个整数，输出它们的和。

解题思路：

1. 定义两个 `int`
2. `std::cin >> a >> b;`
3. `std::cout` 输出结果

完整代码：

```
#include <iostream>

int main() {
    int a, b;
    std::cout << "请输入两个整数：";
    std::cin >> a >> b;
    std::cout << a << " + " << b << " = " << (a + b) << std::endl;
    return 0;
}
```

要点/易错：

- `cin` 变量前不需要取地址 `&` (与 `scanf` 不同)
- 输入类型不匹配可能导致后续读入失败

八、作业练习

先独立完成，做完再到本节文末「参考答案」对照。

练习1：配置 C++ 编译任务

按本篇配置 VS Code 的 `tasks.json`，确保 `Ctrl+Shift+B` 能成功编译 `.cpp` 文件（也可用命令行完成同等验证）。

练习2：输出个人信息

创建 `info.cpp`，用 `std::cout` 输出姓名、学号、专业、语言等信息，带简单分隔线。

练习3：两数之和（C++ 版）

使用 `std::cin` / `std::cout` 实现两整数求和，并口头对比 `scanf` / `printf`。

挑战题：读入三个 `double`，输出平均值（保留两位小数；可用 `<iomanip>` 的 `std::fixed` 与 `std::setprecision`）。

九、参考答案

练习1 参考答案

思路/解析：确认编译器为 `g++`，参数含 `-std=c++17 -Wall`；命令行等价验证：

```
g++ -std=c++17 -Wall info.cpp -o info
```

练习2 参考答案

```
#include <iostream>

int main() {
    std::cout << "===== 个人信息 =====" << std::endl;
    std::cout << "姓名: 张三" << std::endl;
    std::cout << "学号: 2024001001" << std::endl;
    std::cout << "专业: 计算机科学与技术" << std::endl;
    std::cout << "语言: C++" << std::endl;
    std::cout << "===== " << std::endl;
    return 0;
}
```

练习3 参考答案

思路/解析：`cin` 自动按类型提取，无需格式串与 `&`。

```
#include <iostream>

int main() {
    int a, b;
    std::cin >> a >> b;
    std::cout << a + b << std::endl;
    return 0;
}
```

挑战题参考答案

思路/解析：求和除以 3.0；用 `fixed` + `setprecision(2)` 控制小数位。

```
#include <iostream>
#include <iomanip>

int main() {
    double a, b, c;
    std::cin >> a >> b >> c;
    double avg = (a + b + c) / 3.0;
    std::cout << std::fixed << std::setprecision(2) << avg << std::endl;
    return 0;
}
```

十、总结与延伸

本节重点回顾

1. C++ 与 C 共用工具链，编译器为 g++/clang++
2. VS Code 需将 tasks.json 中的 `gcc` 改为 `g++`，并添加 `-std=c++17`
3. 单文件：`g++ -std=c++17 -Wall file.cpp -o out`
4. 多文件项目推荐使用 CMake

下节预告

下一篇我们将学习《程序结构：头文件、命名空间与编译链接》，理解 C++ 程序的模块组织方式。

延伸阅读

- [C 语言开发环境搭建](#)
- [engineering/02-linux-and-shell/](#) — Linux 下的 GCC/g++ 实践

📌 知识点卡片

- C++ 编译器：g++、clang++；源文件：`.cpp`
- 推荐编译：`g++ -std=c++17 -Wall -g file.cpp -o out`
- CMake 用于多文件项目的跨平台构建
- VS Code：Extension Pack for C++ + tasks.json + launch.json

💡 小技巧

- 始终用 g++ 编译 .cpp，避免用 gcc 导致链接错误
- 养成 `-Wall` 习惯，尽早发现潜在问题

⚠ 注意事项

- C 头文件（如 `stdio.h`）在 C++ 中建议写为 `cstdio`（C++ 风格）
- 调试前必须先编译成功

如果你在配置过程中遇到问题，欢迎在评论区留言！下一篇见！

Guide 3

程序结构：头文件、命名空间与编译链接

C++ 程序的模块化组织

在 [Hello World 背后：C 语言程序的执行原理](#) 中，我们学习了 C 程序从源码到可执行文件的完整流程：预处理 → 编译 → 汇编 → 链接。C++ 程序遵循相同的编译模型，但在**程序组织**上引入了命名空间、更严格的头文件约定等新机制。

本篇在 C 编译原理基础上，重点讲解 C++ 特有的程序结构。

一、C++ 程序的编译流程（回顾）

C++ 与 C 共享相同的编译四阶段：

```
hello.cpp → 预处理 → 编译 → 汇编 → 链接 → 可执行文件
```

g++ 分步命令：

```
g++ -E hello.cpp -o hello.i      # 预处理
g++ -S hello.cpp -o hello.s      # 编译为汇编
g++ -c hello.cpp -o hello.o      # 汇编为目标文件
g++ hello.o -o hello             # 链接
g++ hello.cpp -o hello           # 一步完成
```

内存布局、目标文件结构、静态/动态链接等细节请参阅 [C 语言第 03 篇](#)。

二、头文件与源文件

2.1 C++ 头文件约定

类型	扩展名	作用
源文件	<code>.cpp</code> / <code>.cc</code> / <code>.cxx</code>	存放函数与类的实现
头文件	<code>.h</code> / <code>.hpp</code> / <code>.hxx</code>	存放声明、类定义、模板

C vs C++ 头文件风格：

C 风格	C++ 推荐风格	说明
<code>#include <stdio.h></code>	<code>#include <cstdio></code>	C++ 标准库头文件
<code>#include <stdlib.h></code>	<code>#include <cstdlib></code>	内容在 <code>std</code> 命名空间
<code>#include <iostream></code>	同左	C++ 专用，无 <code>.h</code> 后缀

2.2 头文件保护 (Include Guard)

防止头文件被重复包含：

```
// math_utils.h
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

int add(int a, int b);
int multiply(int a, int b);

#endif // MATH_UTILS_H
```

C++ 现代写法（推荐）：

```
// math_utils.h
#pragma once

int add(int a, int b);
int multiply(int a, int b);
```

2.3 多文件项目示例

math_utils.h:

```
#pragma once

int add(int a, int b);
int multiply(int a, int b);
```

math_utils.cpp:

```
#include "math_utils.h"

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}
```

main.cpp:

```
#include <iostream>
#include "math_utils.h"

int main() {
    std::cout << "3 + 5 = " << add(3, 5) << std::endl;
    std::cout << "3 * 5 = " << multiply(3, 5) << std::endl;
    return 0;
}
```

编译:

```
g++ -std=c++17 -Wall main.cpp math_utils.cpp -o app
./app
```

三、命名空间 (namespace)

3.1 为什么需要命名空间

C 语言中，所有全局符号共享同一命名空间，容易发生命名冲突。C++ 引入 **namespace** 将标识符分组：

```
namespace Math {
    int add(int a, int b) { return a + b; }
}

namespace Utils {
    int add(int a, int b) { return a + b + 1; } // 不同命名空间，不冲突
}

int main() {
    std::cout << Math::add(1, 2) << std::endl; // 3
    std::cout << Utils::add(1, 2) << std::endl; // 4
    return 0;
}
```

3.2 std 命名空间

C++ 标准库的所有内容都在 `std` 命名空间中：

```
#include <iostream>
#include <string>

int main() {
    std::cout << "Hello" << std::endl;
    std::string name = "C++";
    return 0;
}
```

3.3 using 声明与 using 指令

写法	作用	建议
<code>using std::cout;</code>	引入单个名称	函数内可使用
<code>using namespace std;</code>	引入整个命名空间	不推荐 在头文件中使用

```
#include <iostream>

int main() {
    using std::cout;
    using std::endl;
    cout << "Hello" << endl;
    return 0;
}
```

在头文件中写 `using namespace std;` 会污染所有包含该头文件的代码，应尽量避免。

3.4 匿名命名空间

相当于文件内部的 `static`，限制符号仅在当前翻译单元可见：

```
namespace {
    int internal_helper() { return 42; }
}
```

四、预处理指令（C++ 扩展）

C++ 支持 C 的所有预处理指令（`#include`、`#define`、`#ifdef` 等），并增加：

特性	说明	示例
<code>#pragma once</code>	头文件保护（非标准但广泛支持）	见上文
<code>constexpr</code>	编译期常量（C++11）	<code>constexpr int N = 100;</code>
原始字符串	多行字符串字面量	<code>R"(line1\nline2)"</code>

条件编译示例：

```
#ifdef DEBUG
    std::cout << "调试模式" << std::endl;
#endif
```

五、main 函数与程序入口

C++ 的 `main` 函数与 C 类似，但 C++ 允许更多形式：

```
int main() { return 0; } // 无参数
int main(int argc, char* argv[]) { ... } // 命令行参数 (与 C 相同)
```

C++ 中 `main` 必须返回 `int` (C 中 `void main()` 是非标准的，C++ 不允许)。

六、知识点实战例题

例题1：命名空间隔离

题目：定义 `namespace A` 与 `namespace B`，各自有 `void print()`，在 `main` 中分别调用。

解题思路：

1. 同名函数放在不同命名空间避免冲突
2. 调用时写 `A::print()` / `B::print()`

完整代码：

```
#include <iostream>

namespace A {
void print() { std::cout << "A\n"; }
}

namespace B {
void print() { std::cout << "B\n"; }
}

int main() {
    A::print();
    B::print();
    return 0;
}
```

要点/易错：

- 只写 `print()` 会找不到声明或产生歧义
- 头文件中避免 `using namespace std;`

例题2：多文件最小结构

题目: `greet.h` 声明 `sayHello`, `greet.cpp` 定义, `main.cpp` 调用。

解题思路:

1. 头文件放声明 + `#pragma once`
2. 源文件包含头文件并实现
3. 一条命令同时编译多个 `.cpp`

完整代码:

```
// greet.h
#pragma once
#include <string>
void sayHello(const std::string& name);
```

```
// greet.cpp
#include "greet.h"
#include <iostream>
void sayHello(const std::string& name) {
    std::cout << "Hello, " << name << std::endl;
}
```

```
// main.cpp
#include "greet.h"
int main() {
    sayHello("C++");
    return 0;
}
```

要点/易错:

- 链接时两个 `.cpp` 都要参与: `g++ -std=c++17 greet.cpp main.cpp -o app`
- 头文件一般不放函数定义 (inline/模板除外)

七、作业练习

先独立完成, 做完再到本节文末「参考答案」对照。

练习1: 多文件编译

创建 `greet.h` / `greet.cpp` / `main.cpp`, 实现 `void sayHello(const std::string& name);`, 用一条 `g++` 命令生成可执行文件。

练习2: 命名空间实验

定义 `namespace A` 和 `namespace B`, 各自包含同名 `print()`, 在 `main` 中用限定名调用。

练习3: 观察预处理

运行 `g++ -E main.cpp` (可用管道查看前若干行), 观察 `#include` 展开。

挑战题: 用匿名命名空间在单个 `.cpp` 中隐藏一个仅内部使用的辅助函数, 并说明它与 `static` 函数的相似点。

八、参考答案

练习1 参考答案

思路/解析：声明与定义分离；链接阶段合并目标文件。

```
g++ -std=c++17 -Wall greet.cpp main.cpp -o app
```

(源码同例题2。)

练习2 参考答案

```
#include <iostream>
namespace A { void print() { std::cout << "A\n"; } }
namespace B { void print() { std::cout << "B\n"; } }
int main() {
    A::print();
    B::print();
    return 0;
}
```

练习3 参考答案

思路/解析：`-E` 只预处理；展开后会出现大量标准库内容，说明「包含头文件」本质是文本展开。

```
g++ -E main.cpp | more
```

挑战题参考答案

思路/解析：匿名命名空间内的符号具有内部链接，仅当前翻译单元可见。

```
#include <iostream>

namespace {
    void helper() { std::cout << "internal\n"; }
}

int main() {
    helper();
    return 0;
}
```

与文件内 `static void helper()` 类似：都是避免污染其他 `.cpp` 的符号表。

九、总结与延伸

本节重点回顾

1. C++ 编译流程与 C 相同：预处理 → 编译 → 汇编 → 链接
2. 头文件用 `#pragma once` 或 include guard 防止重复包含
3. **namespace** 解决命名冲突；标准库在 `std` 中
4. 避免在头文件中使用 `using namespace std;`
5. 多文件项目：`g++ main.cpp utils.cpp -o app`

下节预告

下一篇我们将学习《数据类型、引用与 auto》，掌握 C++ 在 C 类型系统上的重要扩展。

延伸阅读

- [C 语言程序的执行原理](#)
- [C++ 速查：程序基础](#)

📌 知识点卡片

- 源文件 `.cpp`，头文件 `.h` / `.hpp`；`#pragma once` 防重复包含
- 标准库头文件：`<iostream>`、`<string>` 等，内容在 `std` 命名空间
- 命名空间：`namespace Name { ... }`，访问用 `Name::symbol`
- 多文件编译：`g++ file1.cpp file2.cpp -o app`

💡 小技巧

- C 标准库头文件在 C++ 中用 `<cstdio>` 等形式，符号在 `std` 中
- 用 `g++ -E` 查看预处理结果，理解 `#include` 展开

⚠️ 注意事项

- 头文件中只放声明，实现放在 `.cpp` 中（模板除外，后续章节讲解）
- 不要在头文件写 `using namespace std;`

如果你对程序结构有任何疑问，欢迎在评论区留言！下一篇见！

Guide 4

数据类型、引用与 auto

C++ 在 C 类型系统上的扩展

在 [C 语言数据类型与变量](#) 中，我们学习了 `int`、`float`、`char` 等基本类型。C++ 完全保留了这些类型，并在此基础上增加了 `bool`、引用、`auto` 等重要特性，同时提供了类型安全的 `iostream` 输入输出。

本篇聚焦 C++ 相对 C 的类型与 I/O 扩展；整型、浮点、字符等基础内容请参阅 C 教程。

一、C++ 基本数据类型（回顾与扩展）

1.1 与 C 兼容的基本类型

C++ 支持 C 的所有基本类型：`char`、`short`、`int`、`long`、`float`、`double` 等。用法与 C 相同。

详细说明见 [C 语言数据类型与变量](#)。

1.2 C++ 特有：bool 类型

C++ 内置布尔类型 `bool`，取值为 `true` 或 `false`：

```
#include <iostream>

int main() {
    bool isReady = true;
    bool isEmpty = false;

    std::cout << std::boolalpha; // 输出 true/false 而非 1/0
    std::cout << "isReady: " << isReady << std::endl;
    std::cout << "isEmpty: " << isEmpty << std::endl;

    int score = 85;
    bool passed = (score >= 60);
    std::cout << "passed: " << passed << std::endl;

    return 0;
}
```

对比	C	C++
布尔类型	无（用 int 0/1 模拟）	<code>bool</code> ， <code>true</code> / <code>false</code>
条件判断	非零为真	同左，但 <code>bool</code> 语义更清晰

二、引用 (Reference)

2.1 什么是引用

引用是变量的别名——定义时必须初始化，且之后不能绑定到其他对象：

```
#include <iostream>

int main() {
    int original = 42;
    int& ref = original; // ref 是 original 的别名

    ref = 100;
    std::cout << "original: " << original << std::endl; // 100
    std::cout << "ref: " << ref << std::endl;           // 100

    return 0;
}
```

2.2 引用 vs 指针

特性	引用 <code>int&</code>	指针 <code>int*</code>
初始化	必须初始化	可为空 (NULL/nullptr)
重新绑定	不能	可以指向其他对象
解引用	直接使用, 无需 <code>*</code>	需要 <code>*</code> 解引用
空值	不存在空引用	可以 nullptr
主要用途	函数参数、返回值	动态内存、可选值

```
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 1, y = 2;
    swap(x, y); // 直接修改 x、y, 无需传地址
    std::cout << x << ", " << y << std::endl; // 2, 1
    return 0;
}
```

指针的完整讲解见 [C 语言指针](#)。C++ 中优先用引用传递非空对象，用指针表示可选或动态内存。

2.3 const 引用

不能通过 const 引用修改原对象，常用于函数参数以避免拷贝：

```
void print(const std::string& s) {
    std::cout << s << std::endl;
    // s = "other"; // 错误：不能修改
}
```

三、auto 与类型推导 (C++11)

3.1 auto 关键字

编译器根据初始化表达式自动推导类型：

```
#include <iostream>
#include <vector>

int main() {
    auto i = 42;           // int
    auto d = 3.14;         // double
    auto s = std::string("hello"); // std::string
}
```

```
std::vector<int> vec = {1, 2, 3};
for (auto it = vec.begin(); it != vec.end(); ++it) {
    std::cout << *it << " ";
}
std::cout << std::endl;

return 0;
}
```

3.2 使用建议

场景	是否推荐 auto
迭代器、复杂模板类型	推荐
简单整型、浮点	可选，显式类型更清晰
函数返回值	谨慎，需确保可读性

3.3 decltype

获取表达式的类型，常用于模板元编程：

```
int x = 0;
decltype(x) y = 10; // y 的类型与 x 相同，为 int
```

四、const 与 constexpr

4.1 const

与 C 类似，表示只读：

```
const int MAX_SIZE = 100;
const double PI = 3.14159;
```

4.2 constexpr (C++11)

编译期常量，值在编译时确定：

```
constexpr int BUFFER_SIZE = 256;
constexpr int square(int n) { return n * n; }

int main() {
    int arr[square(5)]; // 数组大小为 25，编译期确定
    return 0;
}
```

五、iostream 输入输出

5.1 标准输出 cout

```
#include <iostream>

int main() {
    int age = 25;
    double height = 1.75;

    std::cout << "年龄: " << age << std::endl;
    std::cout << "身高: " << height << " 米" << std::endl;

    return 0;
}
```

与 printf 对比:

特性	printf	cout
类型安全	需手动匹配格式符	编译期检查
语法	<code>printf("%d", x)</code>	<code>cout << x</code>
扩展性	固定	可重载 <code><<</code>

5.2 标准输入 cin

```
#include <iostream>

int main() {
    int a, b;
    std::cout << "请输入两个整数: ";
    std::cin >> a >> b;
    std::cout << "和为: " << a + b << std::endl;

    return 0;
}
```

C 的 `scanf` / `printf` 在 C++ 中仍可用 (`#include <cstdio>`) , 但推荐优先使用 `iostream`。

5.3 常用操纵符

```
#include <iostream>
#include <iomanip>

int main() {
    double pi = 3.14159265;
    std::cout << std::fixed << std::setprecision(2) << pi << std::endl; // 3.14
    std::cout << std::boolalpha << true << std::endl; // true
    return 0;
}
```

六、类型大小与 sizeof

与 C 相同，`sizeof` 返回类型或变量占用的字节数：

```
#include <iostream>

int main() {
    std::cout << "sizeof(int): " << sizeof(int) << std::endl;
    std::cout << "sizeof(bool): " << sizeof(bool) << std::endl;
    std::cout << "sizeof(double): " << sizeof(double) << std::endl;
    return 0;
}
```

七、知识点实战例题

例题1：引用交换

题目：实现 `void swap(int& a, int& b)`，在 `main` 中验证。

解题思路：

1. 引用参数是实参别名，修改即改原变量
2. 用临时变量完成三点交换

完整代码：

```
#include <iostream>

void swap(int& a, int& b) {
    int t = a;
    a = b;
    b = t;
}

int main() {
    int x = 3, y = 5;
    swap(x, y);
    std::cout << x << " " << y << std::endl; // 5 3
    return 0;
}
```

要点/易错:

- 引用必须绑定到已存在对象, 不能为 null
- 形参写成 `int a` 则只改副本, 无法交换实参

例题2: auto 与四则运算

题目: 用 `auto` 保存中间结果, 读入两个 `double` 并输出它们的和。

解题思路:

1. `cin` 读入
2. `auto sum = a + b;` 由编译器推导
3. `cout` 输出

完整代码:

```
#include <iostream>

int main() {
    double a, b;
    std::cin >> a >> b;
    auto sum = a + b;
    std::cout << sum << std::endl;
    return 0;
}
```

要点/易错:

- `auto` 必须有初始值才能推导
- 整数相除仍是整数除法: `auto x = 1 / 2;` 得到 0

八、作业练习

先独立完成, 做完再到本节文末「参考答案」对照。

练习1: 引用交换 — 编写 `void swap(int& a, int& b)` 并验证。

练习2: auto 推导 — 声明多个 `auto` 变量, 可用 `typeid` 打印类型名 (实现定义, 重在观察差异)。

练习3: `iostream` 计算器 — 读入两个数与运算符 (`+ - * /`), 输出结果 (除法注意除数为 0)。

挑战题: 分别用引用与指针实现交换函数, 并简述二者差异。

九、参考答案

练习1 参考答案

(同例题1代码。)

练习2 参考答案

```
#include <typeinfo>
#include <iostream>

int main() {
    auto x = 10;
    auto y = 3.14;
    std::cout << typeid(x).name() << std::endl;
    std::cout << typeid(y).name() << std::endl;
    return 0;
}
```

练习3 参考答案

思路/解析：先读两个操作数与一个字符运算符，再用 `if` / `switch` 分支；除法前检查 0。

```
#include <iostream>

int main() {
    double a, b;
    char op;
    std::cin >> a >> op >> b;
    if (op == '+') std::cout << a + b;
    else if (op == '-') std::cout << a - b;
    else if (op == '*') std::cout << a * b;
    else if (op == '/') {
        if (b == 0) std::cout << "div0";
        else std::cout << a / b;
    } else std::cout << "bad op";
    std::cout << std::endl;
    return 0;
}
```

挑战题参考答案

思路/解析：引用更简洁；指针可表示「可空」，但调用要取地址。

```
void swap_ref(int& a, int& b) {
    int t = a; a = b; b = t;
}

void swap_ptr(int* a, int* b) {
    int t = *a; *a = *b; *b = t;
}
```

差异：引用必须绑定且不可重新绑定（普通引用）；指针可改变指向、可为 `nullptr`，调用写作 `swap_ptr(&x, &y)`。

十、总结与延伸

本节重点回顾

1. C++ 保留 C 基本类型，新增 **bool** 类型
2. **引用**是别名，必须初始化，常用于函数参数避免拷贝
3. **auto** 让编译器推导类型，简化复杂类型声明
4. **iostream** (cout/cin) 提供类型安全的 I/O，推荐替代 printf/scanf
5. **constexpr** 用于编译期常量与函数

下节预告

下一篇将进入核心语法阶段：《运算符与表达式：C++ 扩展特性》，学习 new/delete 等 C++ 运算符。

延伸阅读

- [C 语言数据类型与变量](#)
- [C 语言指针](#)
- [C++ 速查：数据类型与引用](#)

📌 知识点卡片

- bool: `true` / `false`；引用 `T&` 必须初始化，是别名
- auto: 编译期类型推导；const 引用 `const T&` 用于只读参数
- 输出: `std::cout << x << std::endl`；输入: `std::cin >> x`
- C 基础类型用法见 C 教程；C++ 增量重点在引用、bool、iostream

💡 小技巧

- 函数参数中，大对象用 `const T&` 传递，避免拷贝
- 简单循环变量可写 `for (auto i = 0; i < n; ++i)`

⚠️ 注意事项

- 引用必须初始化，不能为空
- 不要返回局部变量的引用或指针（悬垂引用）
- 混用 cout 与 printf 时注意缓冲区刷新

如果你对数据类型与引用有疑问，欢迎在评论区留言！下一篇见！

Guide 5

运算符与表达式：C++扩展特性

在 C 运算符基础上扩展

在 [C 语言运算符与表达式](#) 中，我们系统学习了算术、关系、逻辑、赋值、位运算等运算符。C++ 完全兼容这些运算符，并新增了 **new/delete**、**nullptr**、流运算符 `<</>>` 等重要特性。

本篇重点讲解 C++ 扩展；算术、关系、逻辑等基础运算符请参阅 C 教程。

一、C 运算符回顾（链接）

C++ 支持 C 的全部运算符：

类别	运算符	详见
算术	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	C 第 05 篇 §二
关系	<code>></code> <code><</code> <code>>=</code> <code><=</code> <code>==</code> <code>!=</code>	C 第 05 篇 §三
逻辑	<code>&&</code> <code> </code> <code>!</code>	同上
赋值	<code>=</code> <code>+=</code> <code>-=</code> 等	同上
位运算	<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code>	同上

C++ 差异：关系与逻辑运算的结果类型为 `bool`（C 中为 `int`）。

二、new 与 delete 运算符

2.1 动态内存分配

C 使用 `malloc` / `free`，C++ 推荐使用 **new/delete**：

```
#include <iostream>

int main() {
    // 分配单个 int
    int* p = new int(42);
    std::cout << *p << std::endl;
    delete p;

    // 分配数组
    int* arr = new int[5]{1, 2, 3, 4, 5};
    for (int i = 0; i < 5; i++)
        std::cout << arr[i] << " ";
    std::cout << std::endl;
    delete[] arr; // 数组必须用 delete[]

    return 0;
}
```

操作	语法	释放
单个对象	<code>new Type(值)</code>	<code>delete ptr;</code>
对象数组	<code>new Type[n]{...}</code>	<code>delete[] ptr;</code>

现代 C++ 更推荐智能指针（见 [第 13 篇](#)），避免裸 new/delete。

2.2 new 与 malloc 的区别

特性	new/delete	malloc/free
类型	类型安全，返回正确类型指针	返回 <code>void*</code> ，需强制转换
构造/析构	调用构造/析构函数	不调用
失败处理	抛出 <code>bad_alloc</code> 异常	返回 NULL
所属	C++ 运算符	C 库函数

三、nullptr

C++11 引入 `nullptr` 作为空指针字面量，替代 C 的 `NULL`（实为 0）：

```
int* p = nullptr;

if (p == nullptr)
    std::cout << "空指针" << std::endl;

// 函数重载时 nullptr 不会歧义
void f(int x)    { std::cout << "int\n"; }
void f(int* p)  { std::cout << "pointer\n"; }
f(nullptr);    // 调用 f(int*)
```

四、流运算符 << 与 >>

4.1 输出运算符 <<

```
#include <iostream>

int main() {
    int x = 10;
    double y = 3.14;
    std::cout << "x = " << x << ", y = " << y << std::endl;
    return 0;
}
```

<< 可链式调用，编译期检查类型。

4.2 输入运算符 >>

```
int a, b;
std::cin >> a >> b;
```

五、作用域解析运算符 ::

```
std::cout << "Hello";    // std 命名空间
Math::add(1, 2);         // 自定义命名空间
Student::getName();       // 类静态成员（后续章节）
```

六、类型转换运算符

C++ 提供四种显式类型转换（C 风格 `(type)expr` 仍可用但不推荐）：

运算符	用途	示例
<code>static_cast</code>	编译期可检查的转换	<code>static_cast<double>(a)</code>
<code>dynamic_cast</code>	多态类型的安全向下转换	见第 11 篇
<code>const_cast</code>	去除/添加 const	少用
<code>reinterpret_cast</code>	低层重新解释位模式	慎用

```
int a = 10;
double d = static_cast<double>(a);
```

七、运算符重载（初识）

C++ 允许为用户定义类型重载运算符（类章节详述）：

```
// 后续在类与对象章节学习
// std::string s1 = "Hello", s2 = " World";
// std::string s3 = s1 + s2; // 运算符+被重载
```

八、知识点实战例题

例题1: new/delete 单个对象

题目：用 `new` 分配一个 `int`，赋值为 42，输出后 `delete`。

解题思路：

- `int* p = new int(42);`
- 使用 `*p`
- `delete p;` 后可置 `p = nullptr;`

完整代码：

```
#include <iostream>

int main() {
    int* p = new int(42);
    std::cout << *p << std::endl;
    delete p;
    p = nullptr;
    return 0;
}
```

要点/易错:

- `new` 与 `delete` 成对; 数组用 `new[]` / `delete[]`
- 忘记 `delete` 会造成泄漏

例题2: nullptr 判断

题目: 编写 `void print(int* p)`, 空指针输出「空」, 否则输出值。

解题思路:

1. 先判断 `p == nullptr`
2. 再解引用

完整代码:

```
#include <iostream>

void print(int* p) {
    if (p == nullptr) std::cout << "空" << std::endl;
    else std::cout << *p << std::endl;
}

int main() {
    int x = 10;
    print(nullptr);
    print(&x);
    return 0;
}
```

要点/易错:

- 优先用 `nullptr`, 不要用 `NULL` / `0` 混用造成重载歧义
- 解引用前必须确认非空

九、作业练习

先独立完成, 做完再到本节文末「参考答案」对照。

练习1: `new/delete` 练习 — 动态分配长度为 `n` 的 `double` 数组, 填入 `1~n` 并输出, 最后 `delete[]`。

练习2: `nullptr` 与指针 — 实现例题2的 `print`。

练习3: 类型转换 — 用 `static_cast<double>` 计算两个 `int` 的真实除法结果。

挑战题：先写一段会泄漏的 `new` 代码，再用 `std::unique_ptr`（或明确写出应 `delete` 的修正版）说明如何避免。

十、参考答案

练习1 参考答案

思路/解析：数组分配用 `new double[n]`，释放必须 `delete[]`。

```
#include <iostream>

int main() {
    int n;
    std::cin >> n;
    double* a = new double[n];
    for (int i = 0; i < n; ++i) a[i] = i + 1;
    for (int i = 0; i < n; ++i) std::cout << a[i] << " ";
    std::cout << std::endl;
    delete[] a;
    return 0;
}
```

练习2 参考答案

（同例题2。）

练习3 参考答案

思路/解析：先把其中一个操作数转为 `double`，再除。

```
#include <iostream>

int main() {
    int a = 5, b = 2;
    double r = static_cast<double>(a) / b;
    std::cout << r << std::endl; // 2.5
    return 0;
}
```

挑战题参考答案

思路/解析：裸 `new` 在中途 `return` 时易泄漏；智能指针在析构时自动释放。

```
#include <memory>

void leaky() {
    int* p = new int(1);
    // 若此处提前 return, 则泄漏
    delete p;
}

void safe() {
    auto p = std::make_unique<int>(1);
} // 自动释放
```

十一、总结与延伸

本节重点回顾

1. C++ 兼容 C 全部运算符，关系/逻辑结果为 `bool`
2. **new/delete** 用于动态内存，数组用 `delete[]`
3. **nullptr** 是类型安全的空指针
4. `<</>` 用于 iostream 输入输出
5. `::` 用于命名空间与类成员访问

下节预告

下一篇：《流程控制》，在 C 基础上学习 bool 条件与 C++ 风格写法。

延伸阅读

- [C 语言运算符与表达式](#)
- [C++ 速查 §3 运算符](#)

📌 知识点卡片

- new/delete：类型安全的动态内存；delete[] 释放数组
- nullptr 替代 NULL；static_cast 用于安全类型转换
- << >> 流运算符；:: 作用域解析

⚠️ 注意事项

- new 与 delete 必须配对，new[] 与 delete[] 必须配对
- 避免裸 new/delete，优先使用 vector 和智能指针

有疑问欢迎在评论区留言！下一篇见！

Guide 6

流程控制：让程序学会做选择

C++ 中的条件分支

C++ 的流程控制语法与 C 完全相同：`if-else`、`switch`、三目运算符 `? :`。本篇采用混合定位——简要回顾并标注 C++ 差异，详细语法与示例请参阅 [C 语言流程控制](#)。

一、if 语句（与 C 相同）

```
#include <iostream>

int main() {
    int score = 85;

    if (score >= 90)
        std::cout << "优秀" << std::endl;
    else if (score >= 60)
        std::cout << "及格" << std::endl;
    else
        std::cout << "不及格" << std::endl;

    return 0;
}
```

C++ 差异：

- 条件表达式可结果为 `bool` 类型
- 推荐用 `bool` 变量：`bool passed = (score >= 60);`

完整 if 语法、嵌套、常见错误见 [C 第 06 篇](#)。

二、switch 语句（与 C 相同）

```
#include <iostream>

int main() {
    char op = '+';
    int a = 10, b = 3, result;

    switch (op) {
        case '+': result = a + b; break;
        case '-': result = a - b; break;
        case '*': result = a * b; break;
        default:
            std::cout << "无效运算符" << std::endl;
            return 1;
    }
    std::cout << "结果: " << result << std::endl;
    return 0;
}
```

2.1 C++11: enum class 与 switch

C++ 强类型枚举可与 switch 配合，避免隐式转换：

```
enum class Color { Red, Green, Blue };

Color c = Color::Red;
switch (c) {
    case Color::Red:    std::cout << "红\n"; break;
    case Color::Green:  std::cout << "绿\n"; break;
    case Color::Blue:   std::cout << "蓝\n"; break;
}
```

三、三目运算符

与 C 相同，C++14 起可与 `auto` 配合：

```
int a = 10, b = 20;
auto max = (a > b) ? a : b;
std::cout << max << std::endl;
```

四、C++ 风格初始化与条件

4.1 列表初始化 (C++11)

```
int x{10};           // 直接初始化
int arr[]{1, 2, 3};  // 数组列表初始化
```

4.2 bool 与条件

```
bool flag = true;
if (flag) { /* ... */ }

// 避免 if (x = 5) 误写为赋值
if (int x = getValue(); x > 0) { // C++ 条件内声明 (作用域限于 if 块)
    std::cout << x << std::endl;
}
```

五、知识点实战例题

例题1：成绩等级

题目：输入分数，输出 A/B/C/D/F，并用 `bool` 表示是否及格 (≥ 60)。

解题思路：

1. 用 if-else 分段

2. `bool pass = score >= 60;`

完整代码：

```
#include <iostream>

int main() {
    int score;
    std::cin >> score;
    bool pass = score >= 60;
    char grade;
    if (score >= 90) grade = 'A';
    else if (score >= 80) grade = 'B';
    else if (score >= 70) grade = 'C';
    else if (score >= 60) grade = 'D';
    else grade = 'F';
    std::cout << grade << (pass ? " pass" : " fail") << std::endl;
    return 0;
}
```

要点/易错：

- 分支顺序要从高分到低分
- 分数范围校验可按需添加

例题2: enum class 菜单

题目：定义 `enum class Menu { Add, Delete, Query, Exit }`，读入整数映射到枚举并 `switch`。

解题思路：

1. 强类型枚举需 `static_cast`
2. `switch` 覆盖各分支

完整代码：

```
#include <iostream>

enum class Menu { Add = 1, Delete, Query, Exit };

int main() {
    int x;
    std::cin >> x;
    auto m = static_cast<Menu>(x);
    switch (m) {
        case Menu::Add: std::cout << "Add\n"; break;
        case Menu::Delete: std::cout << "Delete\n"; break;
        case Menu::Query: std::cout << "Query\n"; break;
        case Menu::Exit: std::cout << "Exit\n"; break;
        default: std::cout << "Unknown\n"; break;
    }
    return 0;
}
```


要点/易错:

- `enum class` 不能隐式转 `int`; 比较要用 `Menu::Add`
- `switch` 漏 `break` 会贯穿

六、作业练习

先独立完成，做完再到本节文末「参考答案」对照。

练习1: 成绩等级 — 用 `if-else` + `bool` 及格标记。

练习2: `enum class` 菜单 — 定义菜单枚举并用 `switch` 处理。

练习3: 三目运算符 — 求两数最大值，结果存 `auto`。

挑战题: 用 `enum class` + `do-while` + `switch` 写一个简易 ATM 菜单（查询/存款/取款/退出），余额用变量模拟即可。

七、参考答案

练习1 参考答案

（同例题1，可按评分标准微调分数线。）

练习2 参考答案

（同例题2。）

练习3 参考答案

```
#include <iostream>

int main() {
    int a, b;
    std::cin >> a >> b;
    auto m = (a > b) ? a : b;
    std::cout << m << std::endl;
    return 0;
}
```

挑战题参考答案

思路/解析：循环直到选择退出；各分支修改 `balance`。

```
#include <iostream>

enum class ATM { Query = 1, Deposit, Withdraw, Exit };

int main() {
    double balance = 1000;
    int x;
    do {
        std::cout << "1查 2存 3取 4退: ";
        std::cin >> x;

        auto op = static_cast<ATM>(x);
```

```

switch (op) {
    case ATM::Query:
        std::cout << balance << std::endl;
        break;
    case ATM::Deposit: {
        double v; std::cin >> v; balance += v; break;
    }
    case ATM::Withdraw: {
        double v; std::cin >> v;
        if (v <= balance) balance -= v;
        else std::cout << "insufficient\n";
        break;
    }
    case ATM::Exit: break;
    default: std::cout << "bad\n"; break;
}
} while (static_cast<ATM>(x) != ATM::Exit);
return 0;
}

```

八、总结与延伸

本节重点回顾

1. if/switch/三目运算符与 C 语法相同
2. C++ 条件结果类型为 `bool`；推荐用 `bool` 变量
3. **enum class** 强类型枚举，配合 switch 更安全
4. C++11 支持条件内声明变量、列表初始化

下节预告

下一篇：《循环结构》，学习传统循环与 范围 for。

延伸阅读

- [C 语言流程控制 \(完整版\)](#)
- [C++ 速查 §4 流程控制](#)

📌 知识点卡片

- if/switch 与 C 相同；条件表达式结果为 bool
- enum class 强类型枚举；switch case 需加 break
- 三目： `cond ? a : b`；可与 auto 配合

下一篇见！

Guide 7

循环结构：重复执行的艺术

传统循环与范围 for

C++ 支持 C 的全部循环：for、while、do-while。C++11 新增 **范围 for (range-based for)**，简化容器与数组遍历。本篇在 C 循环基础上重点讲解范围 for；传统循环详见 [C 语言循环结构](#)。

一、for 循环（与 C 相同）

```
#include <iostream>

int main() {
    for (int i = 0; i < 10; i++)
        std::cout << i << " ";
    std::cout << std::endl;
    return 0;
}
```

完整 for/while/do-while 语法、break/continue、嵌套循环见 [C 第 07 篇](#)。

二、范围 for (C++11)

2.1 基本语法

```
for (declaration : range)
    statement;
```

遍历数组：

```
int arr[] = {1, 2, 3, 4, 5};
for (int x : arr)
    std::cout << x << " ";
```

遍历 vector：

```
#include <vector>
std::vector<int> vec = {10, 20, 30};
for (int v : vec)
    std::cout << v << " ";
```

2.2 三种遍历方式

形式	语法	说明
值拷贝	<code>for (auto x : container)</code>	修改 x 不影响原容器
引用	<code>for (auto& x : container)</code>	可修改元素
常引用	<code>for (const auto& x : container)</code>	只读，避免拷贝（推荐）

```
#include <iostream>
#include <vector>
#include <string>

int main() {
    std::vector<std::string> names = {"Alice", "Bob", "Charlie"};

    // 只读遍历 (推荐)
    for (const auto& name : names)
        std::cout << name << " ";
    std::cout << std::endl;

    // 修改元素
    for (auto& name : names)
        name += "!";
    for (const auto& name : names)
        std::cout << name << " ";
    std::cout << std::endl;

    return 0;
}
```

2.3 遍历 string

```
#include <string>
std::string s = "Hello";
for (char c : s)
    std::cout << c;
```

三、while 与 do-while

与 C 完全相同：

```
int n = 0;
while (n < 5) {
    std::cout << n << " ";
    n++;
}

do {
    std::cout << "至少执行一次\n";
} while (false);
```

四、break 与 continue

与 C 相同：跳出循环 / 跳过本次迭代。

五、知识点实战例题

例题1：范围 for 求和

题目：vector 存 1~10，用范围 for 求和。

解题思路：

1. push_back 或构造时填充
2. for (int x : v) sum += x;

完整代码：

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    for (int i = 1; i <= 10; ++i) v.push_back(i);
    int sum = 0;
    for (int x : v) sum += x;
    std::cout << sum << std::endl;
    return 0;
}
```

要点/易错：

- 只要只读，可用 const auto&
- 不要在范围 for 中随意增删正在遍历的容器

例题2：引用修改元素

题目：将 vector 中所有元素翻倍。

解题思路：

1. 使用 auto& 才能改原元素
2. 若写 auto x 则改的是副本

完整代码：

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{1, 2, 3};
    for (auto& x : v) x *= 2;
    for (int x : v) std::cout << x << " ";
    std::cout << std::endl;
    return 0;
}
```

要点/易错：

- `auto` 与 `auto&` 差别是本题核心

六、作业练习

先独立完成，做完再到本节文末「参考答案」对照。

练习1：范围 for 求和 — `vector` 存 1~100 并求和。

练习2：修改元素 — `for (auto& x : vec)` 翻倍。

练习3：嵌套循环 — 打印 5×5 乘法表 (`i * j`，*i, j* 从 1 到 5)。

挑战题：打印 1~9 的完整九九乘法表。

七、参考答案

练习1 参考答案

思路/解析：先填充再累加；也可用公式 $100 \times 101 / 2$ 验算。

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    for (int i = 1; i <= 100; ++i) v.push_back(i);
    long long sum = 0;
    for (int x : v) sum += x;
    std::cout << sum << std::endl;
    return 0;
}
```

练习2 参考答案

(同例题2。)

练习3 参考答案

```
#include <iostream>

int main() {
    for (int i = 1; i <= 5; ++i) {
        for (int j = 1; j <= 5; ++j)
            std::cout << i * j << "\t";
        std::cout << std::endl;
    }
    return 0;
}
```

挑战题参考答案

思路/解析：内层循环到 `i` 即可得到下三角九九表；完整矩形则内层到 9。

```
#include <iostream>

int main() {
    for (int i = 1; i <= 9; ++i) {
        for (int j = 1; j <= i; ++j)
            std::cout << j << "*" << i << "=" << i * j << "\t";
        std::cout << std::endl;
    }
    return 0;
}
```

八、总结与延伸

本节重点回顾

1. for/while/do-while 与 C 相同
2. 范围 for: `for (auto x : range)` 简化遍历
3. 大对象用 `const auto&` 避免拷贝
4. break/continue 用法与 C 相同

下节预告

下一篇：《数组、string 与容器初探》，学习 `std::string` 和 `std::vector`。

延伸阅读

- [C 语言循环结构（完整版）](#)
- [C++ 速查 §4.3 范围 for](#)

📌 知识点卡片

- 范围 for: `for (const auto& x : container)`
- 修改元素用 `auto&`；只读用 `const auto&`
- 传统 for/while/do-while 与 C 相同

下一篇见!

Guide 8

数组、string与容器初探

从 C 数组到 C++ 容器

在 [C 语言数组与字符串](#) 中，我们学习了 C 数组和字符数组。C++ 保留 C 数组语法，并提供了 `std::string` 和 `std::vector` 等更安全、易用的容器。

一、C 风格数组（回顾）

C++ 完全支持 C 数组：

```
int arr[5] = {1, 2, 3, 4, 5};
int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};

for (int i = 0; i < 5; i++)
    std::cout << arr[i] << " ";
```

数组定义、遍历、作为参数传递详见 [C 第 08 篇](#)。

C++11 列表初始化：

```
int arr[]{1, 2, 3}; // 自动推断大小
```

二、std::string

2.1 基本用法

```
#include <iostream>
#include <string>

int main() {
    std::string s1 = "Hello";
    std::string s2("World");
    std::string s3 = s1 + ", " + s2 + "!";

    std::cout << s3 << std::endl;
    std::cout << "长度: " << s3.length() << std::endl;

    // 输入
    std::string name;
    std::cout << "请输入姓名: ";

    std::cin >> name;
```



```
std::cout << "你好, " << name << std::endl;

return 0;
}
```

2.2 常用操作

操作	方法/语法	示例
长度	<code>size()</code> / <code>length()</code>	<code>s.size()</code>
拼接	<code>+</code> 或 <code>+=</code>	<code>s += "!"</code>
访问	<code>[i]</code> 或 <code>at(i)</code>	<code>s[0]</code> (at 有边界检查)
子串	<code>substr(pos, len)</code>	<code>s.substr(0, 5)</code>
查找	<code>find(str)</code>	<code>s.find("lo")</code>
比较	<code>==</code> <code>!=</code> <code><</code> <code>></code>	<code>s1 == s2</code>

2.3 与 C 字符串对比

特性	C <code>char[]</code>	C++ <code>std::string</code>
长度	需手动管理	自动
拼接	<code>strcat</code>	<code>+</code> / <code>+=</code>
内存	固定或动态	自动扩展
安全性	易越界	<code>at()</code> 可检查

三、std::vector

3.1 基本用法

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec = {1, 2, 3, 4, 5};

    vec.push_back(6);      // 尾部添加
    vec.pop_back();        // 删除尾部

    std::cout << "大小: " << vec.size() << std::endl;
    std::cout << "第一个: " << vec.front() << std::endl;

    std::cout << "最后一个: " << vec.back() << std::endl;
}
```

```
for (const auto& v : vec)
    std::cout << v << " ";
std::cout << std::endl;

return 0;
}
```

3.2 常用操作

操作	方法	说明
添加	<code>push_back(x)</code>	尾部添加
删除	<code>pop_back()</code>	删除尾部
大小	<code>size()</code>	元素个数
清空	<code>clear()</code>	删除所有元素
访问	<code>[i]</code> / <code>at(i)</code>	下标访问
空判断	<code>empty()</code>	是否为空

3.3 vector 与数组

特性	C 数组	vector
大小	编译期固定	运行期可变
内存	栈或手动分配	自动管理
传递	退化为指针	可按值/引用传递

四、std::array (C++11, 可选)

固定大小数组的 C++ 封装：

```
#include <array>
std::array<int, 5> arr = {1, 2, 3, 4, 5};
std::cout << arr.size() << std::endl; // 5
```

五、知识点实战例题

例题1: string 拼接与查找

题目：构造字符串 `Hello`，拼接 `C++`，查找子串 `C++` 的位置并输出。

解题思路:

1. `s += "..."` 或 `s = s + ...`
2. `s.find("C++")` 返回下标或 `npos`

完整代码:

```
#include <iostream>
#include <string>

int main() {
    std::string s = "Hello";
    s += " C++";
    auto pos = s.find("C++");
    if (pos != std::string::npos)
        std::cout << pos << " " << s << std::endl;
    return 0;
}
```

要点/易错:

- `find` 失败返回 `std::string::npos`, 不要当成普通下标直接用

例题2: vector 动态添加

题目: 读入若干整数, 以 0 结束, 输出全部元素。

解题思路:

1. 循环 `cin >> x`, `x==0` 结束
2. `push_back` 后范围 for 输出

完整代码:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    int x;
    while (std::cin >> x && x != 0) v.push_back(x);
    for (int n : v) std::cout << n << " ";
    std::cout << std::endl;
    return 0;
}
```

要点/易错:

- 0 作为结束哨兵通常不入队

六、作业练习

先独立完成, 做完再到本节文末「参考答案」对照。

练习1: string 操作 — 用 `getline` 读入一行, 统计单词数 (以空格分隔) 或反转字符串 (二选一或都做)。

练习2: vector 动态添加 — 读到 0 结束并输出。

练习3: 找最大值 — 读入 n 与 n 个整数，输出最大值及其下标（从 0 开始）。

挑战题: 统计字符串中元音字母 (aeiouAEIOU) 个数。

七、参考答案

练习1 参考答案（反转）

思路/解析：也可用双指针；这里演示 `std::reverse`。

```
#include <iostream>
#include <string>
#include <algorithm>

int main() {
    std::string line;
    std::getline(std::cin, line);
    std::reverse(line.begin(), line.end());
    std::cout << line << std::endl;
    return 0;
}
```

练习2 参考答案

（同例题2。）

练习3 参考答案

思路/解析：边读边维护最大值与下标。

```
#include <iostream>
#include <vector>

int main() {
    int n;
    std::cin >> n;
    std::vector<int> v(n);
    for (int i = 0; i < n; ++i) std::cin >> v[i];
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (v[i] > v[best]) best = i;
    std::cout << v[best] << " at " << best << std::endl;
    return 0;
}
```

挑战题参考答案

```
#include <iostream>
#include <string>

int main() {
    std::string s;
```

```
std::getline(std::cin, s);
int cnt = 0;
for (char c : s) {
    if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u' ||
        c=='A' || c=='E' || c=='I' || c=='O' || c=='U')
        ++cnt;
}
std::cout << cnt << std::endl;
return 0;
}
```

八、总结与延伸

本节重点回顾

1. C 数组在 C++ 中仍可用；推荐用 `std::string` 和 `std::vector`
2. `string`：自动管理长度，`+` 拼接，`size()` 求长
3. `vector`：动态数组，`push_back` / `pop_back`，范围 `for` 遍历
4. 大容器遍历时用 `const auto&` 避免拷贝

下节预告

下一篇：《函数：重载、默认参数与内联》。

延伸阅读

- [C 语言数组与字符串](#)
- [C++ 速查 S7 数组与 string](#)

📌 知识点卡片

- `std::string`：`#include`；`+` 拼接；`size()` 长度
- `std::vector`：`#include`；`push_back`；范围 `for` 遍历
- `getline` 读整行；`at()` 有边界检查

下一篇见！

Guide 9

函数：重载、默认参数与内联

C++ 函数的扩展能力

在 [C 语言函数](#) 中，我们学习了函数定义、参数传递、返回值与递归。C++ 在此基础上增加了 **函数重载**、**默认参数**、**inline** 和 **函数模板**，使函数更加灵活强大。

一、C 函数基础（链接）

C++ 完全支持 C 风格函数：定义、调用、值传递、指针传递、返回值、递归等。

详见 [C 第 09 篇](#)。

C++ 推荐：大对象用 `const T&` 传递，避免拷贝（见 [第 04 篇](#)）。

二、函数重载

同一作用域内，函数名相同但**参数列表不同**（类型或个数），编译器根据实参选择匹配版本：

```
#include <iostream>

int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}

int main() {
    std::cout << add(1, 2) << std::endl;    // 调用 int 版
    std::cout << add(1.5, 2.5) << std::endl; // 调用 double 版
    std::cout << add(1, 2, 3) << std::endl;  // 调用三参数版
    return 0;
}
```

重载规则：

- 仅返回值不同**不能**构成重载
 - 参数个数或类型必须不同
 - C 不支持重载，C++ 支持
-

三、默认参数

函数参数可指定默认值，调用时可省略：

```
#include <iostream>

void greet(const std::string& name, const std::string& prefix = "Hello") {
    std::cout << prefix << ", " << name << "!" << std::endl;
}

int main() {
    greet("Alice");           // Hello, Alice!
    greet("Bob", "Hi");       // Hi, Bob!
    return 0;
}
```

规则：

- 默认参数从右向左连续： `void f(int a, int b = 0, int c = 0);` 合法
- 默认参数在**声明**中指定（定义中可省略）
- 默认参数与重载配合使用需谨慎，避免歧义

四、inline 函数

`inline` 建议编译器将函数体 inline 展开，减少调用开销（适用于短小的频繁调用函数）：

```
inline int square(int x) {
    return x * x;
}
```

- 定义通常放在头文件中
- 现代编译器会自动 inline，显式 `inline` 更多是 ODR 相关
- 类内定义的成员函数默认为 inline

五、函数模板（初探）

编写与类型无关的函数，编译器根据实参类型生成具体版本：

```
#include <iostream>

template<typename T>
T maximum(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    std::cout << maximum(3, 5) << std::endl;           // int
    std::cout << maximum(3.14, 2.71) << std::endl;     // double
    return 0;
}
```

模板详解见 [第 12 篇](#)。

六、constexpr 函数 (C++11)

编译期可求值的函数：

```
constexpr int factorial(int n) {  
    return (n <= 1) ? 1 : n * factorial(n - 1);  
}  
  
int arr[factorial(5)]; // 120 元素数组，编译期确定
```

七、知识点实战例题

例题1：重载 print

题目：实现 `print(int)`、`print(double)`、`print(const std::string&)`。

解题思路：

1. 同名不同参数列表
2. 调用时由实参类型决议

完整代码：

```
#include <iostream>  
#include <string>  
  
void print(int x) { std::cout << "int " << x << std::endl; }  
void print(double x) { std::cout << "double " << x << std::endl; }  
void print(const std::string& s) { std::cout << "string " << s << std::endl; }  
  
int main() {  
    print(1);  
    print(1.5);  
    print(std::string("hi"));  
    return 0;  
}
```

要点/易错：

- 仅返回类型不同不能构成重载
- 字面量 `"hi"` 是 `const char*`，可能更匹配别的重载；用 `std::string` 或再加重载

例题2：默认参数

题目：`void draw(char ch = '*', int count = 5)` 打印若干字符。

解题思路：

1. 默认参数写在声明处，从右到左
2. 调用可省略右侧参数

完整代码：


```
#include <iostream>

void draw(char ch = '*', int count = 5) {
    for (int i = 0; i < count; ++i) std::cout << ch;
    std::cout << std::endl;
}

int main() {
    draw();
    draw('#');
    draw('-', 10);
    return 0;
}
```

要点/易错:

- 不能隔开省略中间参数
- 声明与定义不要重复写两套默认值

八、作业练习

先独立完成，做完再到本节文末「参考答案」对照。

练习1: 重载 print — 三个重载版本。

练习2: 默认参数 — 实现 `draw`。

练习3: 函数模板 — `template<typename T> void swap(T& a, T& b)`。

挑战题: 写一个函数模板 `T mymax(T a, T b)`，并说明它与针对具体类型的重载 `max` 如何共存、如何被选中。

九、参考答案

练习1 参考答案

(同例题1。)

练习2 参考答案

(同例题2。)

练习3 参考答案

```
template<typename T>
void swap(T& a, T& b) {
    T t = a; a = b; b = t;
}
```

挑战题参考答案

思路/解析：模板可匹配广泛类型；若存在更匹配的非模板重载，优先普通函数。

```
#include <iostream>

template<typename T>
```

```
T mymax(T a, T b) { return a > b ? a : b; }

int mymax(int a, int b) { // 非模板重载
    return a > b ? a : b;
}

int main() {
    std::cout << mymax(3, 5) << std::endl; // 倾向调用 int 重载
    std::cout << mymax(3.1, 2.0) << std::endl; // 模板 double
    return 0;
}
```

十、总结与延伸

本节重点回顾

1. **函数重载**：同名不同参，C 不支持
2. **默认参数**：从右向左，声明处指定
3. **inline**：建议内联，短小函数适用
4. **函数模板**：`template<typename T>` 泛型编程入门

下节预告

进入进阶阶段：《类与对象：面向对象基础》。

延伸阅读

- [C 语言函数](#)
- [C++ 速查 §5 函数与重载](#)

📌 知识点卡片

- 重载：参数类型/个数不同；仅返回值不同无效
- 默认参数：`void f(int a, int b = 0);`
- `template T func(T x);`

下一篇见！

Guide 10

类与对象：面向对象基础

用类组织数据与行为

C 语言用结构体 + 函数组织数据，C++ 的 **类 (class)** 将数据与操作数据的函数封装在一起，是面向对象编程的基础。

一、类与对象的概念

- **类 (class)**：自定义类型的模板，描述对象的属性和行为
- **对象 (object)**：类的实例，占用内存的具体实体

```
#include <iostream>
#include <string>

class Student {
public:
    std::string name;
    int age;

    void introduce() {
        std::cout << "我是 " << name << ", 今年 " << age << " 岁" << std::endl;
    }
};

int main() {
    Student s1;
    s1.name = "张三";
    s1.age = 20;
    s1.introduce();
    return 0;
}
```

二、访问控制

关键字	含义	典型用途
public	公有，内外均可访问	对外接口
private	私有，仅类内可访问	隐藏实现细节
protected	保护，类内和派生类可访问	继承时使用

```
class BankAccount {
private:
    double balance;

public:
    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
    double getBalance() const {
        return balance;
    }
};
```

封装：将数据设为 private，通过 public 成员函数访问，保护数据完整性。

三、构造函数与析构函数

3.1 构造函数

对象创建时自动调用，用于初始化：

```
class Rectangle {
private:
    double width, height;

public:
    Rectangle(double w, double h) : width(w), height(h) {}

    double area() const {
        return width * height;
    }
};

int main() {
    Rectangle rect(3.0, 4.0);
    std::cout << rect.area() << std::endl; // 12
    return 0;
}
```

初始化列表： `: width(w), height(h)` 在构造函数体执行前初始化成员，效率更高。

3.2 析构函数

对象销毁时自动调用，用于释放资源：

```
class FileHandler {
public:
    FileHandler() { std::cout << "打开文件\n"; }
    ~FileHandler() { std::cout << "关闭文件\n"; }
};

// 离开作用域时自动调用 ~FileHandler()
```

四、this 指针

成员函数内隐含指向当前对象的指针 `this`：

```
class Counter {
private:
    int count;

public:
    Counter() : count(0) {}
    Counter& increment() {
        this->count++;
        return *this; // 支持链式调用
    }
    int getCount() const { return count; }
};
```

五、const 成员函数

不修改对象状态的成员函数声明为 const:

```
class Point {
private:
    int x, y;
public:
    Point(int x, int y) : x(x), y(y) {}
    int getX() const { return x; } // 不修改成员
    void setX(int nx) { x = nx; }
};
```

const 对象只能调用 const 成员函数。

六、静态成员

属于类而非某个对象，所有对象共享：

```
class Counter {
private:
    static int total;

public:
    Counter() { total++; }
    static int getTotal() { return total; }
};

int Counter::total = 0; // 类外定义静态成员
```

七、知识点实战例题

例题1: Circle 类

题目：含半径、构造函数、`area()`、`circumference()`。

解题思路：

1. 成员变量可 `private`
2. 面积 `pi*r*r`，周长 `2*pi*r`

完整代码：

```
#include <iostream>
#include <cmath>

class Circle {
    double r;
public:
    explicit Circle(double radius) : r(radius) {}
    double area() const { return 3.141592653589793 * r * r; }
    double circumference() const { return 2 * 3.141592653589793 * r; }
};

int main() {
    Circle c(2.0);
    std::cout << c.area() << " " << c.circumference() << std::endl;
    return 0;
}
```

要点/易错：

- 构造函数初始化列表更清晰
- `const` 成员函数承诺不修改对象

例题2：封装 Temperature

题目：`private` 存摄氏度；提供摄氏读写与华氏读取。

解题思路：

1. 华氏 = 摄氏 * 9/5 + 32
2. setter 可做范围检查（可选）

完整代码：

```
class Temperature {
    double celsius_;
public:
    void setCelsius(double c) { celsius_ = c; }
    double getCelsius() const { return celsius_; }
    double getFahrenheit() const { return celsius_ * 9.0 / 5.0 + 32.0; }
};
```

要点/易错：

- 不要把内部表示直接暴露为 `public` 字段

八、作业练习

先独立完成，做完再到本节文末「参考答案」对照。

练习1：Circle 类 — 半径、构造、area / circumference。

练习2：封装 — Temperature 类。

练习3：链式调用 — StringBuilder::append 返回 *this。

挑战题：实现 Counter，构造归零，inc() 返回 Counter& 支持 c.inc().inc()，并提供 get()。

九、参考答案

练习1 参考答案

(同例题1。)

练习2 参考答案

(同例题2，可补 main 测试。)

练习3 参考答案

```
#include <iostream>
#include <string>

class StringBuilder {
    std::string data_;
public:
    StringBuilder& append(const std::string& s) {
        data_ += s;
        return *this;
    }
    const std::string& str() const { return data_; }
};

int main() {
    StringBuilder sb;
    sb.append("a").append("b");
    std::cout << sb.str() << std::endl;
    return 0;
}
```

挑战题参考答案

```
class Counter {
    int n_ = 0;
public:
    Counter& inc() { ++n_; return *this; }
    int get() const { return n_; }
};
```

十、总结与延伸

本节重点回顾

1. **class** 封装数据（成员变量）与行为（成员函数）
2. **public/private** 访问控制实现封装
3. **构造函数**初始化，**析构函数**清理；初始化列表 `: member(val)`
4. **this** 指向当前对象；**const** 成员函数不修改状态

下节预告

下一篇：《继承、多态与虚函数》。

延伸阅读

- [C 语言结构体](#)
- [C++ 速查 §8 类与对象](#)

📌 知识点卡片

- `class Name { public: ... private: ... };`
- 构造：`Class(args) : member(val) {}`
- 析构：`~Class() {}`
- `const` 成员函数：`void f() const;`

下一篇见！

Guide 11

继承、多态与虚函数

代码复用与运行时绑定

继承让派生类获得基类的成员；**多态**让同一接口在不同对象上表现出不同行为。C++ 通过 **virtual** 虚函数实现运行时多态。

一、继承基础

1.1 语法

```
#include <iostream>
#include <string>

class Person {
protected:
    std::string name;
    int age;
```



```
public:
    Person(const std::string& n, int a) : name(n), age(a) {}

    void showInfo() const {
        std::cout << name << ", " << age << " 岁" << std::endl;
    }
};

class Student : public Person {
private:
    std::string studentId;

public:
    Student(const std::string& n, int a, const std::string& id)
        : Person(n, a), studentId(id) {}

    void showInfo() const {
        Person::showInfo();
        std::cout << "学号: " << studentId << std::endl;
    }
};

int main() {
    Student s("李四", 20, "2024001");
    s.showInfo();
    return 0;
}
```

1.2 继承方式

方式	含义
<code>public</code> 继承	最常用, is-a 关系
<code>protected</code> 继承	较少用
<code>private</code> 继承	实现继承, 较少用

二、访问控制与继承

基类成员	<code>public</code> 继承	<code>protected</code> 继承	<code>private</code> 继承
<code>public</code>	<code>public</code>	<code>protected</code>	<code>private</code>
<code>protected</code>	<code>protected</code>	<code>protected</code>	<code>private</code>
<code>private</code>	不可访问	不可访问	不可访问

派生类可访问基类的 `public` 和 `protected` 成员。

三、虚函数与多态

3.1 问题：非虚函数

```
class Animal {
public:
    void speak() { std::cout << "..." << std::endl; }
};

class Dog : public Animal {
public:
    void speak() { std::cout << "汪汪! " << std::endl; }
};

Animal* p = new Dog();
p->speak(); // 输出 "..." - 非多态!
delete p;
```

3.2 虚函数解决

```
class Animal {
public:
    virtual void speak() { std::cout << "..." << std::endl; }
    virtual ~Animal() {} // 虚析构，多态删除时必须
};

class Dog : public Animal {
public:
    void speak() override { std::cout << "汪汪! " << std::endl; }
};

Animal* p = new Dog();
p->speak(); // 输出 "汪汪! " - 多态!
delete p;
```

virtual：通过基类指针/引用调用时，根据**实际对象类型**决定调用哪个版本。

3.3 override 与 final (C++11)

```
void speak() override; // 显式标记重写
void speak() final;     // 禁止进一步重写
```

四、纯虚函数与抽象类

含纯虚函数的类为**抽象类**，不能实例化：

```
class Shape {
public:
```

```

    virtual double area() const = 0; // 纯虚函数
    virtual ~Shape() = default;
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() const override {
        return 3.14159 * radius * radius;
    }
};

```

五、多态的应用

```

void printArea(const Shape& s) {
    std::cout << "面积: " << s.area() << std::endl;
}

Circle c(5);
Rectangle r(3, 4);
printArea(c); // 圆面积
printArea(r); // 矩形面积

```

同一函数 `printArea` 处理不同形状，体现多态。

六、知识点实战例题

例题1：简单继承

题目： `Animal` 基类， `Dog` 派生并输出不同叫声。

解题思路：

1. `class Dog : public Animal`
2. 派生类可调用基类接口

完整代码：

```

#include <iostream>
#include <string>

class Animal {
public:
    explicit Animal(std::string n) : name(std::move(n)) {}
    void info() const { std::cout << name << std::endl; }
protected:
    std::string name;
};

```

```

class Dog : public Animal {
public:
    explicit Dog(std::string n) : Animal(std::move(n)) {}
    void speak() const { std::cout << name << ": wang\n"; }
};

int main() {
    Dog d("Buddy");
    d.info();
    d.speak();
    return 0;
}

```

要点/易错:

- 继承方式常用 `public`
- 基类构造要在派生类初始化列表中调用

例题2: 虚函数多态

题目: 基类指针调用派生类重写的 `speak`。

解题思路:

1. 基类声明 `virtual`
2. 派生类 `override`
3. 通过基类指针/引用调用

完整代码:

```

#include <iostream>

class Animal {
public:
    virtual void speak() const { std::cout << "... \n"; }
    virtual ~Animal() = default;
};

class Cat : public Animal {
public:
    void speak() const override { std::cout << "meow\n"; }
};

int main() {
    Animal* p = new Cat();
    p->speak();
    delete p;
    return 0;
}

```

要点/易错:

- 多态删除需要虚析构
- 没有 `virtual` 则绑定静态类型

七、作业练习

先独立完成，做完再到本节文末「参考答案」对照。

练习1：继承层次 — `Vehicle` / `Car` / `Bicycle`，各自 `move()`。

练习2：虚函数多态 — `Employee::salary()` 虚函数，`Manager` / `Developer` 重写。

练习3：抽象类 — `Animal` 纯虚 `speak()`，`Cat` / `Dog` 实现，基类指针数组调用。

挑战题：抽象类 `Shape` 纯虚 `area()`，实现 `Circle` 与 `Rectangle`。

八、参考答案

练习1 参考答案

```
#include <iostream>

class Vehicle {
public:
    virtual void move() const { std::cout << "move\n"; }
    virtual ~Vehicle() = default;
};

class Car : public Vehicle {
public:
    void move() const override { std::cout << "drive\n"; }
};

class Bicycle : public Vehicle {
public:
    void move() const override { std::cout << "pedal\n"; }
};
```

练习2 参考答案

```
class Employee {
public:
    virtual double salary() const { return 0; }
    virtual ~Employee() = default;
};

class Manager : public Employee {
public:
    double salary() const override { return 20000; }
};

class Developer : public Employee {
public:
    double salary() const override { return 15000; }
};
```

练习3 参考答案

```
#include <iostream>

class Animal {
public:
    virtual void speak() const = 0;
    virtual ~Animal() = default;
};

class Cat : public Animal {
public:
    void speak() const override { std::cout << "meow\n"; }
};

class Dog : public Animal {
public:
    void speak() const override { std::cout << "wang\n"; }
};

int main() {
    Animal* arr[2] = { new Cat(), new Dog() };
    for (auto* p : arr) { p->speak(); delete p; }
    return 0;
}
```

挑战题参考答案

```
class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
    double r;
public:
    explicit Circle(double R) : r(R) {}
    double area() const override { return 3.14159 * r * r; }
};

class Rectangle : public Shape {
    double w, h;
public:
    Rectangle(double W, double H) : w(W), h(H) {}
    double area() const override { return w * h; }
};
```

九、总结与延伸

本节重点回顾

1. 继承: `class Derived : public Base`

2. **virtual** 实现运行时多态；基类指针调用派生类方法
3. **override** 标记重写；**纯虚函数** `= 0` 定义抽象类
4. 多态基类需**虚析构函数**

下节预告

下一篇：《模板与 STL 基础》。

延伸阅读

- [C++ 速查 §9 继承与多态](#)

📌 知识点卡片

- 继承：`class D : public B;`
- `virtual void f(); override;`
- 纯虚：`virtual void f() = 0;`
- 多态基类：`virtual ~Base() {}`

下一篇见！

Guide 12

模板与STL基础

泛型编程与标准库

模板让代码与类型无关；**STL** (Standard Template Library) 提供容器、算法和迭代器，是 C++ 最强大的特性之一。

一、函数模板

```
#include <iostream>

template<typename T>
T maximum(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    std::cout << maximum(3, 5) << std::endl;
    std::cout << maximum(3.14, 2.71) << std::endl;
    return 0;
}
```

- `template<typename T>` 或 `template<class T>` 等价

- 编译器根据实参类型自动生成 `maximum<int>`、`maximum<double>` 等

二、类模板

```
#include <iostream>

template<typename T>
class Box {
private:
    T content;

public:
    Box(T c) : content(c) {}
    T get() const { return content; }
    void set(T c) { content = c; }
};

int main() {
    Box<int> intBox(42);
    Box<std::string> strBox("Hello");
    std::cout << intBox.get() << std::endl;
    std::cout << strBox.get() << std::endl;
    return 0;
}
```

三、STL 容器

3.1 vector（动态数组）

```
#include <vector>
std::vector<int> vec = {1, 2, 3};
vec.push_back(4);
vec.pop_back();
vec.size();
vec[i];
```

3.2 map（键值对）

```
#include <map>
#include <string>

std::map<std::string, int> scores;
scores["Alice"] = 95;
scores["Bob"] = 87;

for (const auto& pair : scores)
    std::cout << pair.first << ": " << pair.second << std::endl;
```


3.3 set (有序集合)

```
#include <set>
std::set<int> s = {3, 1, 4, 1, 5}; // 自动去重排序
for (int x : s) std::cout << x << " "; // 1 3 4 5
```

四、迭代器

容器提供迭代器遍历：

```
std::vector<int> vec = {1, 2, 3, 4, 5};
for (auto it = vec.begin(); it != vec.end(); ++it)
    std::cout << *it << " ";

// 更简洁：范围 for
for (const auto& v : vec)
    std::cout << v << " ";
```

五、STL 算法

```
#include <algorithm>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> vec = {5, 2, 8, 1, 9};

    std::sort(vec.begin(), vec.end());

    auto it = std::find(vec.begin(), vec.end(), 8);
    if (it != vec.end())
        std::cout << "找到 8, 位置: " << (it - vec.begin()) << std::endl;

    int count = std::count(vec.begin(), vec.end(), 2);
    std::cout << "2 出现 " << count << " 次" << std::endl;

    return 0;
}
```

算法	作用
<code>sort</code>	排序
<code>find</code>	查找元素
<code>count</code>	计数
<code>reverse</code>	反转
<code>for_each</code>	对每个元素执行函数

六、常用头文件

头文件	内容
<code><vector></code>	动态数组
<code><map></code>	键值映射
<code><set></code>	有序集合
<code><string></code>	字符串
<code><algorithm></code>	算法
<code><iostream></code>	输入输出

七、知识点实战例题

例题1：函数模板 swap

题目：编写 `template<typename T> void myswap(T& a, T& b)`。

解题思路：

- 1. 模板头 + 普通函数体
- 2. 调用时自动推导 `T`

完整代码：

```
#include <iostream>

template<typename T>
void myswap(T& a, T& b) {
    T t = a; a = b; b = t;
}

int main() {
    int x = 1, y = 2;
    myswap(x, y);
    std::cout << x << y << std::endl;
    return 0;
}
```

要点/易错:

- 模板通常放在头文件
- 类型须支持拷贝/赋值

例题2: map 计数

题目: 对若干单词统计出现次数 (可写死输入)。

解题思路:

1. `map<string,int>`
2. `freq[word]++`

完整代码:

```
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> freq;
    for (auto w : {"a", "b", "a"}) freq[w]++;
    for (const auto& [k, v] : freq)
        std::cout << k << " " << v << std::endl;
    return 0;
}
```

要点/易错:

- `operator[]` 在键不存在时会插入 0
- C++17 结构化绑定 `[k,v]` 需启用 C++17

八、作业练习

先独立完成, 做完再到本节文末「参考答案」对照。

练习1: 模板栈 — `Stack<T>` 含 `push` / `pop` / `top` / `empty` (可用 `vector` 实现)。

练习2: map 词频 — 读入一行文本统计单词。

练习3: algorithm 综合 — `sort` + `find` + `count`。

挑战题: 对 `vector` 排序后用 `unique` + `erase` 去重。

九、参考答案

练习1 参考答案

```
#include <vector>
#include <stdexcept>

template<typename T>
class Stack {
    std::vector<T> data_;
public:
    void push(const T& x) { data_.push_back(x); }
    void pop() {
        if (data_.empty()) throw std::runtime_error("empty");
        data_.pop_back();
    }
    T& top() { return data_.back(); }
    bool empty() const { return data_.empty(); }
};
```

练习2 参考答案

思路/解析: `istringstream` 按空白拆词。

```
#include <iostream>
#include <map>
#include <sstream>
#include <string>

int main() {
    std::string line;
    std::getline(std::cin, line);
    std::istringstream iss(line);
    std::map<std::string, int> freq;
    std::string w;
    while (iss >> w) freq[w]++;
    for (const auto& p : freq)
        std::cout << p.first << " " << p.second << std::endl;
    return 0;
}
```

练习3 参考答案

```
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{3, 1, 2, 2};
    std::sort(v.begin(), v.end());
    auto it = std::find(v.begin(), v.end(), 2);
    std::cout << (it != v.end() ? "found" : "no") << std::endl;
    std::cout << std::count(v.begin(), v.end(), 2) << std::endl;
    return 0;
}
```

挑战题参考答案

```
#include <algorithm>
#include <vector>

void dedup(std::vector<int>& v) {
    std::sort(v.begin(), v.end());
    v.erase(std::unique(v.begin(), v.end()), v.end());
}
```

十、总结与延伸

本节重点回顾

1. 函数模板 `template<typename T>` 泛型函数
2. 类模板 `template<typename T> class Box`
3. STL 容器: vector、map、set
4. STL 算法: sort、find、count 等, 需 `#include <algorithm>`

下节预告

进入实战阶段:《智能指针与 RAII》。

延伸阅读

- [fundamentals/01-data-structures/](#)
- [C++ 速查 §10~12](#)

📌 知识点卡片

- template 函数/类模板
- vector/map/set; #include
- sort(v.begin(), v.end()); find(v.begin(), v.end(), x);

下一篇见!

Guide 13

智能指针与RAII

自动管理资源

C 语言中动态内存需手动 `malloc / free`，容易泄漏或重复释放。C++ 的 **RAII** (Resource Acquisition Is Initialization) 原则：资源在对象构造时获取，析构时自动释放。**智能指针**是 RAII 的典型应用。

C 内存管理详见 [C 语言内存管理](#)。

一、RAII 原则

```
class FileGuard {
    FILE* fp;
public:
    FileGuard(const char* path) { fp = fopen(path, "r"); }
    ~FileGuard() { if (fp) fclose(fp); }
    FILE* get() { return fp; }
};

void process() {
    FileGuard guard("data.txt");
    // 使用 guard.get() ...
} // 离开作用域, 自动 fclose
```

核心：用对象生命周期管理资源，无需手动释放。

二、unique_ptr

独占所有权，不可复制，可移动：

```
#include <iostream>
#include <memory>

int main() {
    std::unique_ptr<int> p = std::make_unique<int>(42);
    std::cout << *p << std::endl;

    // std::unique_ptr<int> p2 = p; // 错误: 不可复制
    std::unique_ptr<int> p2 = std::move(p); // 可移动

    return 0;
} // 自动 delete
```

操作	说明
<code>std::make_unique<T>(args)</code>	创建 <code>unique_ptr</code> (C++14)
<code>*p</code> / <code>p-></code>	解引用
<code>p.get()</code>	获取裸指针
<code>p.release()</code>	释放所有权，返回裸指针

三、shared_ptr

共享所有权，引用计数：

```
#include <iostream>
#include <memory>

int main() {
    std::shared_ptr<int> p1 = std::make_shared<int>(42);
    std::shared_ptr<int> p2 = p1; // 引用计数 = 2

    std::cout << *p1 << ", 引用计数: " << p1.use_count() << std::endl;

    p2.reset(); // 引用计数 = 1
    return 0;
} // p1 销毁时 delete
```

四、weak_ptr

配合 `shared_ptr` 使用，不增加引用计数，避免循环引用：

```
std::shared_ptr<int> sp = std::make_shared<int>(42);
std::weak_ptr<int> wp = sp;

if (auto locked = wp.lock())
    std::cout << *locked << std::endl;
```

五、与 C malloc/free 对比

特性	malloc/free	智能指针
释放	手动 free	自动析构
异常安全	易泄漏	RAII 保证
所有权	不明确	unique/shared 明确
推荐	C 代码	现代 C++

六、知识点实战例题

例题1: unique_ptr 管理 int

题目：用 `make_unique` 创建 `int`，输出值。

解题思路：

1. `#include <memory>`
2. 离开作用域自动释放

完整代码：

```
#include <iostream>
#include <memory>

int main() {
    auto p = std::make_unique<int>(42);
    std::cout << *p << std::endl;
    return 0;
}
```

要点/易错：

- `unique_ptr` 不可拷贝，可移动
- 优先 `make_unique` 而非裸 `new`

例题2: shared_ptr use_count

题目：拷贝 `shared_ptr` 后观察 `use_count()`。

解题思路：

1. `make_shared`
2. 赋值/拷贝使计数 +1

完整代码：


```
#include <iostream>
#include <memory>

int main() {
    auto a = std::make_shared<int>(1);
    std::cout << a.use_count() << std::endl; // 1
    auto b = a;
    std::cout << a.use_count() << std::endl; // 2
    return 0;
}
```

要点/易错:

- 循环引用会导致无法释放, 需了解 `weak_ptr` (进阶)

七、作业练习

先独立完成, 做完再到本节文末「参考答案」对照。

练习1: `unique_ptr` 管理数组 — 使用 `unique_ptr<int[]>` 或 `unique_ptr` + 自定义删除器 / C++14 数组形式, 分配 `n` 个 `int` 并赋值输出。

练习2: `shared_ptr` 共享 — 两个指针共享同一对象, 打印 `use_count`。

练习3: RAII 文件类 — 构造 `fopen`, 析构 `fclose` (或用 `fstream` 演示 RAII 思想)。

挑战题: 写工厂函数 `std::unique_ptr<int> make_answer()` 返回堆上对象。

八、参考答案

练习1 参考答案

```
#include <iostream>
#include <memory>

int main() {
    int n = 5;
    std::unique_ptr<int[]> a(new int[n]);
    for (int i = 0; i < n; ++i) a[i] = i;
    for (int i = 0; i < n; ++i) std::cout << a[i] << " ";
    std::cout << std::endl;
    return 0;
}
```

练习2 参考答案

(同例题2。)

练习3 参考答案

```
#include <cstdio>

class FileRAII {
    FILE* fp_;
public:
    FileRAII(const char* path, const char* mode) : fp_(std::fopen(path, mode)) {}
    ~FileRAII() { if (fp_) std::fclose(fp_); }
    FILE* get() const { return fp_; }
    FileRAII(const FileRAII&) = delete;
    FileRAII& operator=(const FileRAII&) = delete;
};
```

挑战题参考答案

```
#include <memory>

std::unique_ptr<int> make_answer() {
    return std::make_unique<int>(42);
}
```

九、总结与延伸

本节重点回顾

1. **RAII**: 资源随对象生命周期自动管理
2. **unique_ptr**: 独占所有权, 不可复制
3. **shared_ptr**: 共享所有权, 引用计数
4. **make_unique/make_shared**: 推荐创建方式

下节预告

下一篇: 《综合实战: 面向对象学生管理系统》。

延伸阅读

- [C 语言内存管理](#)
- [C++ 速查 §14 智能指针](#)

知识点卡片

. include

- unique_ptr: 独占; shared_ptr: 共享
- make_unique() / make_shared()

下一篇见!

Guide 14

综合实战：面向对象学生管理系统

将 OOP 与 STL 融会贯通

在前面的章节中，我们学习了类、继承、vector、文件流等知识。本篇综合运用这些特性，实现一个**学生管理系统**：增删改查、数据持久化。

项目结构参考 [C 语言简易计算器](#)。

一、需求分析

功能	说明
添加学生	输入姓名、学号、年龄
删除学生	按学号删除
查询学生	按学号或姓名查询
显示全部	列出所有学生
保存/加载	数据写入/从文件读取

二、设计思路



三、核心代码

3.1 Student 类 (student.h)

```
#pragma once
#include <iostream>
#include <string>

class Student {
public:
    std::string name;
    std::string id;
    int age;

    Student() : age(0) {}
    Student(const std::string& n, const std::string& i, int a)
        : name(n), id(i), age(a) {}

    void display() const {
        std::cout << "学号: " << id << " 姓名: " << name
            << " 年龄: " << age << std::endl;
    }
};
```

3.2 StudentManager 类 (student_manager.h)

```
#pragma once
#include "student.h"
#include <vector>
#include <string>

class StudentManager {
    std::vector<Student> students;

public:
    void addStudent(const Student& s);
    bool removeStudent(const std::string& id);
    Student* findById(const std::string& id);
    void displayAll() const;
    bool saveToFile(const std::string& filename) const;
    bool loadFromFile(const std::string& filename);
};
```

3.3 实现要点 (student_manager.cpp)

```
#include "student_manager.h"
#include <fstream>
#include <algorithm>

void StudentManager::addStudent(const Student& s) {
    students.push_back(s);
}
```

```

bool StudentManager::removeStudent(const std::string& id) {
    auto it = std::remove_if(students.begin(), students.end(),
        [&](const Student& s) { return s.id == id; });
    if (it == students.end()) return false;
    students.erase(it, students.end());
    return true;
}

Student* StudentManager::findById(const std::string& id) {
    for (auto& s : students)
        if (s.id == id) return &s;
    return nullptr;
}

void StudentManager::displayAll() const {
    for (const auto& s : students) s.display();
}

bool StudentManager::saveToFile(const std::string& filename) const {
    std::ofstream ofs(filename);
    if (!ofs) return false;
    for (const auto& s : students)
        ofs << s.id << " " << s.name << " " << s.age << "\n";
    return true;
}

bool StudentManager::loadFromFile(const std::string& filename) {
    std::ifstream ifs(filename);
    if (!ifs) return false;
    students.clear();
    Student s;
    while (ifs >> s.id >> s.name >> s.age)
        students.push_back(s);
    return true;
}

```

3.4 主程序 (main.cpp)

```

#include "student_manager.h"
#include <iostream>

int main() {
    StudentManager mgr;
    mgr.loadFromFile("students.txt");

    int choice;
    do {
        std::cout << "\n1.添加 2.删除 3.查询 4.显示全部 5.保存 0.退出\n";
        std::cout << "选择: ";
        std::cin >> choice;

        if (choice == 1) {
            Student s;

```

```
        std::cout << "姓名 学号 年龄: ";
        std::cin >> s.name >> s.id >> s.age;
        mgr.addStudent(s);
    } else if (choice == 2) {
        std::string id;
        std::cout << "学号: ";
        std::cin >> id;
        std::cout << (mgr.removeStudent(id) ? "已删除\n" : "未找到\n");
    } else if (choice == 3) {
        std::string id;
        std::cout << "学号: ";
        std::cin >> id;
        if (auto* p = mgr.findById(id)) p->display();
        else std::cout << "未找到\n";
    } else if (choice == 4) {
        mgr.displayAll();
    } else if (choice == 5) {
        mgr.saveToFile("students.txt");
        std::cout << "已保存\n";
    }
} while (choice != 0);

return 0;
}
```

四、编译运行

```
g++ -std=c++17 -Wall main.cpp student_manager.cpp -o student_mgr
./student_mgr
```

完整示例见本篇下文代码。

五、扩展建议

- 输入验证（学号唯一性、年龄范围）
- 按姓名模糊查询
- 使用 JSON 或 CSV 格式存储
- 添加单元测试

六、知识点实战例题

例题1：按学号查找

题目：在已有 `StudentManager::findById` 思路下，写一段调用示例：找到则 `display()`，否则提示未找到。

解题思路：

1. 用指针或可选返回值表达「可能不存在」
2. 调用前确保已 `loadFromFile` 或已 `addStudent`

完整代码（示意）：

```
Student* p = mgr.findById("2024001");
if (p) p->display();
else std::cout << "未找到\n";
```

要点/易错：

- 返回内部元素指针时，注意对象生命周期（容器重分配后可能失效）
- 查不到要处理空指针，避免解引用崩溃

七、作业练习

先独立完成，做完再到本节文末「参考答案」对照。

练习1：按姓名查询 — 为 `StudentManager` 增加 `Student* findByName(const std::string& name);`（精确匹配即可），并在菜单中调用。

练习2：学生人数统计 — 提供 `size()`（或等价接口），在菜单中显示当前学生人数。

挑战题：学号唯一性 — `addStudent` 时若学号已存在则拒绝添加并提示。

八、参考答案

练习1 参考答案

思路/解析：线性扫描 `students`，匹配 `name` 后返回地址。

```
Student* StudentManager::findByName(const std::string& name) {
    for (auto& s : students)
        if (s.name == name) return &s;
    return nullptr;
}
```

练习2 参考答案

思路/解析：直接返回容器大小。

```
std::size_t StudentManager::size() const {
    return students.size();
}
```

菜单中可输出：`std::cout << "人数: " << mgr.size() << std::endl;`

挑战题参考答案

思路/解析：添加前先 `findById`，已存在则返回 `false`。

```
bool StudentManager::addStudent(const Student& s) {
    if (findById(s.id)) {
        std::cout << "学号已存在\n";
        return false;
    }
    students.push_back(s);
    return true;
}
```

(若你现有 `addStudent` 为 `void`，可改为返回 `bool`，并同步修改 `main` 中的调用。)

九、总结与延伸

本节重点回顾

1. **Student** 类封装学生数据
2. **StudentManager** 用 `vector` 管理集合，提供 CRUD
3. **fstream** 实现文件持久化
4. 多文件项目：头文件声明 + 源文件实现

下节预告

下一篇：《C++ 学习路线与资源推荐》。

延伸阅读

- [C 语言文件操作](#)
- [本篇综合实战代码](#)

📌 知识点卡片

- `vector` 存储对象；algorithm `remove_if` 删除
- `ofstream` 写文件；`ifstream` 读文件
- 多文件：g++ main.cpp utils.cpp -o app

下一篇见！

Guide 15

C++学习路线与资源推荐

从入门到进阶

完成本专栏 15 篇教程后，你已掌握 C++ 核心语法、OOP、STL 和 RAII。本篇梳理后续学习路径与优质资源。

可参考 [C 语言学习路线与资源推荐](#) 的复习方法。

一、本专栏回顾

阶段	内容	篇目
入门基础	C++ 与 C 关系、环境、程序结构、类型与引用	01 ~ 04
核心语法	运算符、流程控制、循环、string/vector、函数	05 ~ 09
进阶概念	类与对象、继承多态、模板与 STL	10 ~ 12
实战应用	智能指针、学生管理系统、学习路线	13 ~ 15

建议先修：[C 语言教程](#) 01 ~ 09 篇。

二、现代 C++ 标准演进

标准	年份	重要特性
C++98/03	1998	STL、模板、异常
C++11	2011	auto、范围 for、智能指针、lambda、move
C++14	2014	make_unique、泛型 lambda
C++17	2017	structured binding、optional、filesystem
C++20	2020	concepts、ranges、coroutines

推荐：以 **C++17** 为日常开发标准，逐步了解 C++20 特性。

三、进阶学习方向

3.1 深入语言

本专栏 → Effective C++ → Effective Modern C++ → C++ Concurrency in Action

3.2 系统与性能

C++ → 操作系统 → 网络编程 → 高性能计算

3.3 应用开发

C++ → Qt/GTK → 游戏引擎 (Unreal/Unity C++) → 嵌入式

3.4 与 fundamentals 配合

- [数据结构](#)：用 STL 实现，含排序与查找

四、推荐资源

4.1 权威参考

资源	说明	链接
cppreference	C++ 标准库权威参考	https://en.cppreference.com/
C++ 标准草案	语言规范	https://isocpp.org/

4.2 经典书籍

书名	适合阶段
《C++ Primer》	系统学习
《Effective C++》	进阶规范
《Effective Modern C++》	C++11/14
《STL源码剖析》	深入 STL

4.3 在线资源

- learncpp.com — 免费系统教程
- [Compiler Explorer](#) — 在线查看汇编

五、常见问题

Q1: C 和 C++ 都要学吗？

A: 建议先学 C 理解底层，再学 C++ 掌握抽象。本仓库 [C 教程](#) 与 [C++ 教程](#) 可衔接学习。

Q2: 如何避免 C 风格写 C++？

A: 优先用 vector/string 替代数组和 char*；用智能指针替代 new/delete；用 RAII 管理资源。

Q3: 学完本专栏能做什么？

A: 可编写中小型 C++ 程序、理解 OOP 与 STL、阅读开源 C++ 代码入门。

六、知识点实战例题

例题1: 现代特性小 demo

题目：写一段短程序，同时使用 `auto`、范围 for 与 `std::vector`。

解题思路：

1. 用 `vector` 存数据
2. `auto` 推导元素或迭代相关类型
3. 范围 `for` 遍历输出

完整代码：

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{1, 2, 3, 4};
    auto sum = 0;
    for (const auto& x : v) sum += x;
    std::cout << sum << std::endl;
    return 0;
}
```

要点/易错：

- 这是对本专栏现代写法的「体检」，不必引入过多新特性

七、作业练习

先独立完成，做完再到本节文末「参考答案」对照。

练习1：下周学习清单 — 写出 5 条可执行的学习计划（例如：每天刷题库一节 / 读 `cppreference` 某页 / 改写一个小项目）。

练习2：对照 `cppreference` — 查阅 `std::vector::push_back`，写下 3 条笔记（复杂度、是否使迭代器失效、常见用法等）。

挑战题：避免 C 风格 — 把下面 C 风格数组求和改写成 `vector` + 范围 `for`。

```
int a[5] = {1, 2, 3, 4, 5};
int s = 0;
for (int i = 0; i < 5; ++i) s += a[i];
```

八、参考答案

练习1 参考答案

思路/解析：计划要具体、可检查。示例如下（请按自身情况改写）：

1. 复习 Guide 10~12，手写 Circle + 虚函数 Shape
2. 完成题库中「类与对象」主题 10 题
3. 阅读 `cppreference`： `unique_ptr` 页面并做笔记
4. 给学生管理系统加「按姓名查询」
5. 用 Compiler Explorer 对比 `new` 与 `make_unique` 生成代码

练习2 参考答案

思路/解析（示例笔记，查阅后用自己的话重写）：

1. `push_back` 均摊 $O(1)$, 容量不足时触发扩容
2. 扩容会使指向元素的引用/迭代器/指针失效
3. 已知最终大小时可先 `reserve` 减少扩容次数

挑战题参考答案

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> a{1, 2, 3, 4, 5};
    int s = 0;
    for (int x : a) s += x;
    std::cout << s << std::endl;
    return 0;
}
```

九、总结

1. 本专栏覆盖 C++ 核心, 现代特性以 C++11/17 为主
2. 进阶方向: Effective 系列、并发、Qt/游戏/嵌入式
3. 配合 fundamentals 与 engineering 持续实践
4. cppreference 是日常查阅首选

📌 知识点卡片

- 推荐标准: C++17
- 权威参考: cppreference.com
- 进阶书: Effective C++、Effective Modern C++

感谢学习本专栏! 欢迎贡献与反馈。